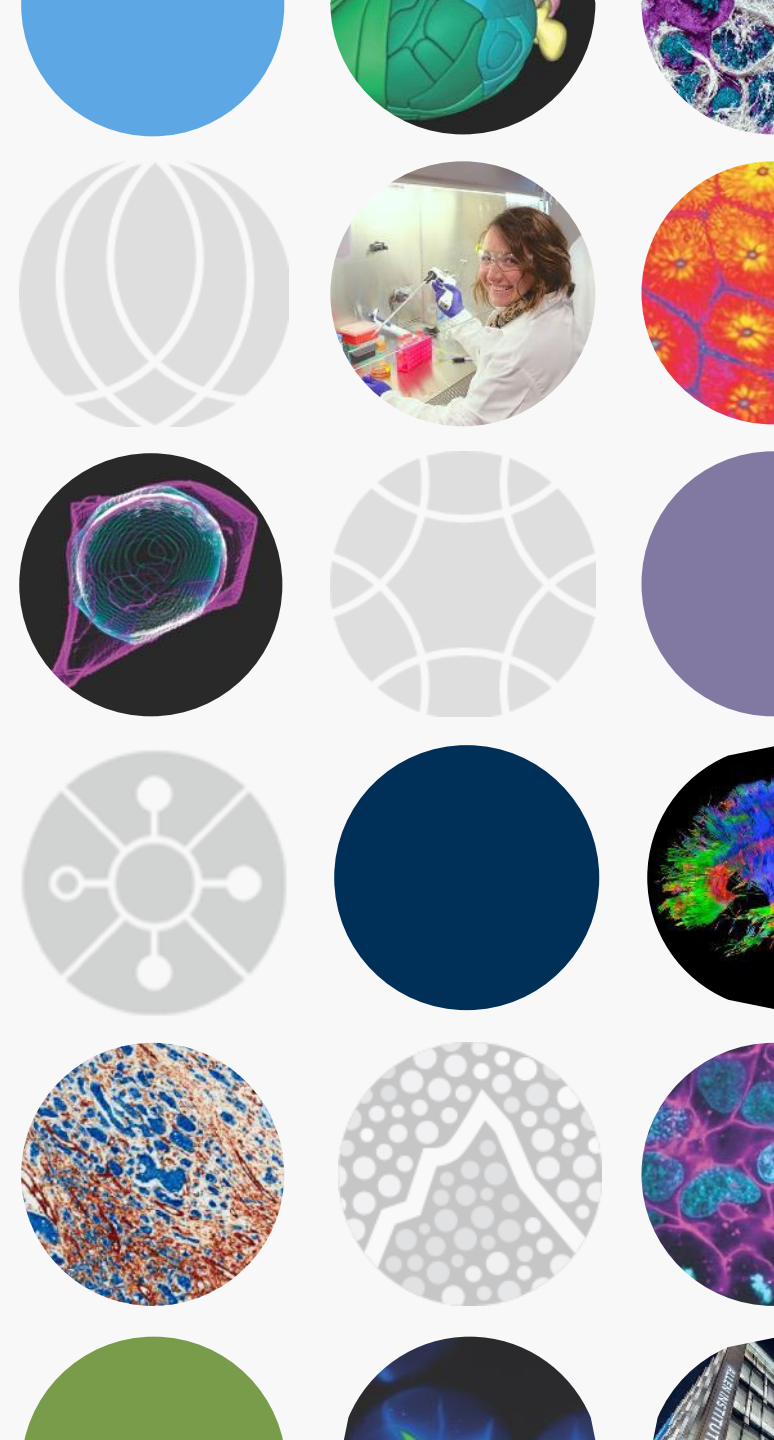




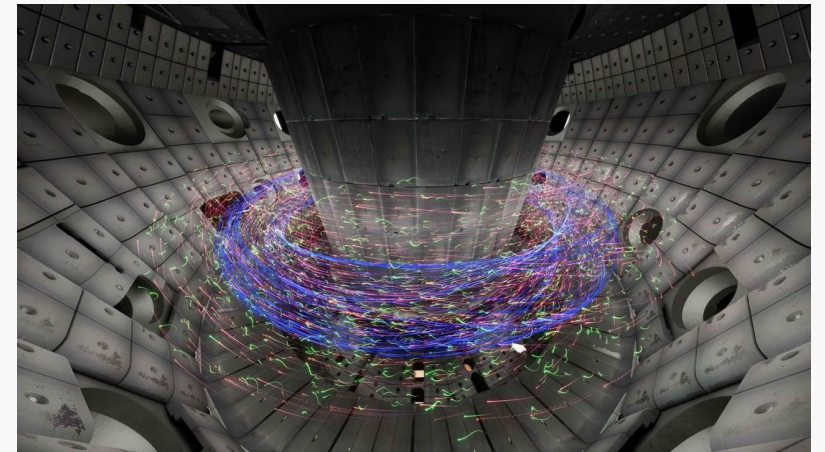
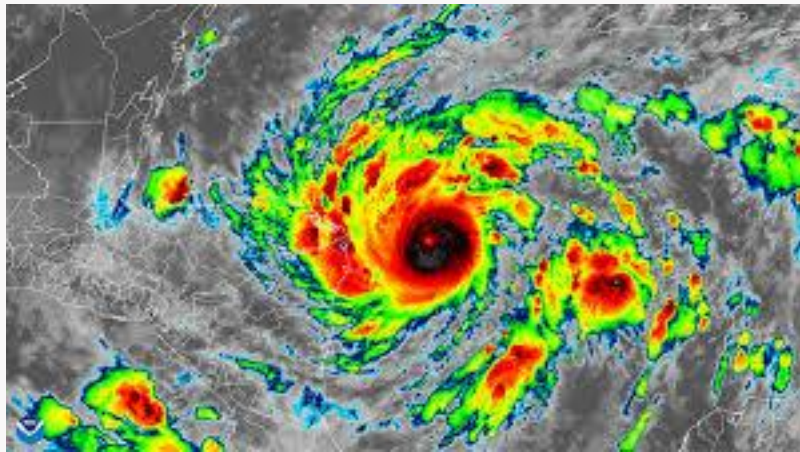
Synchronize or Hop: Two Mechanisms for Predicting the Dynamical World in Modern ML Models

Zhixin Lu

@Santa Fe Institute Seminar
2026

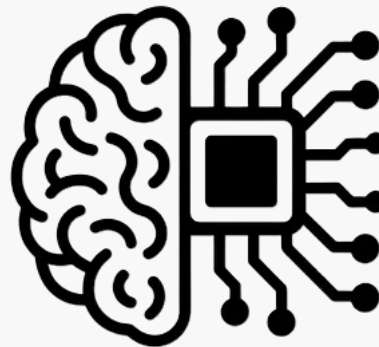


Systems with Complex Dynamics



Modeling

Prediction /
Generation

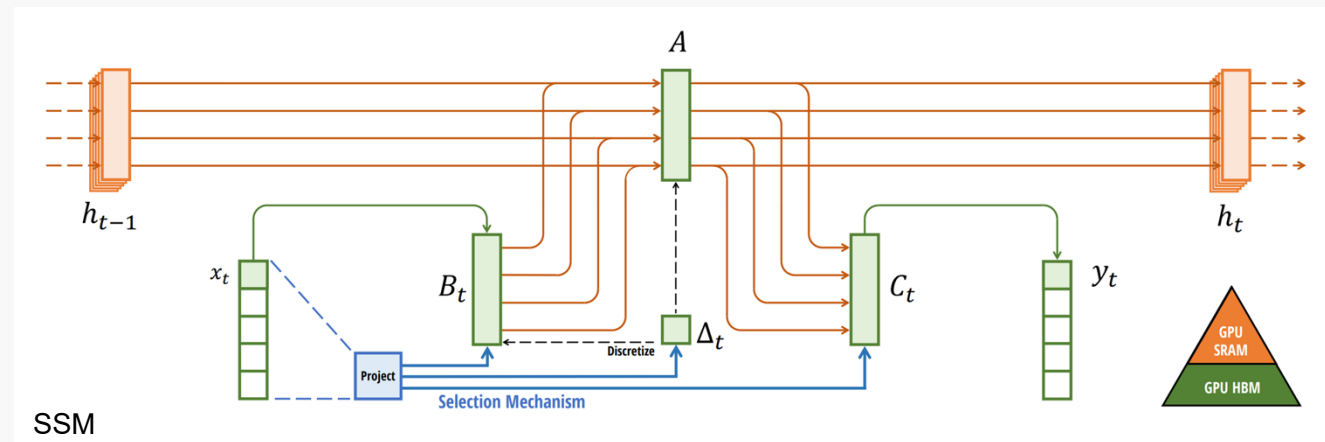
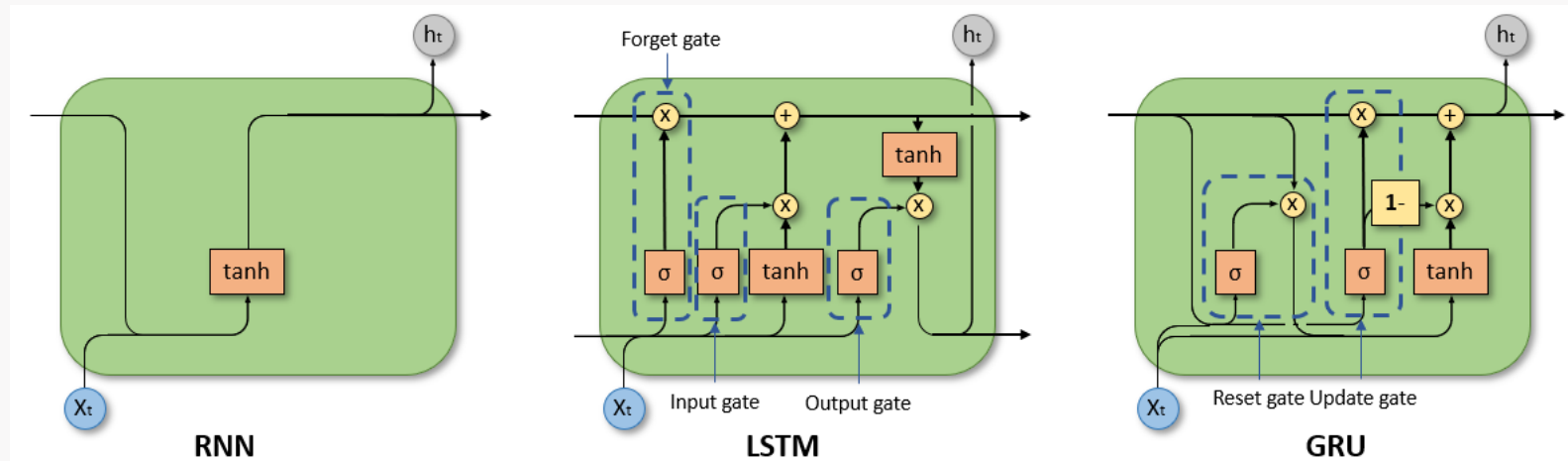


Control

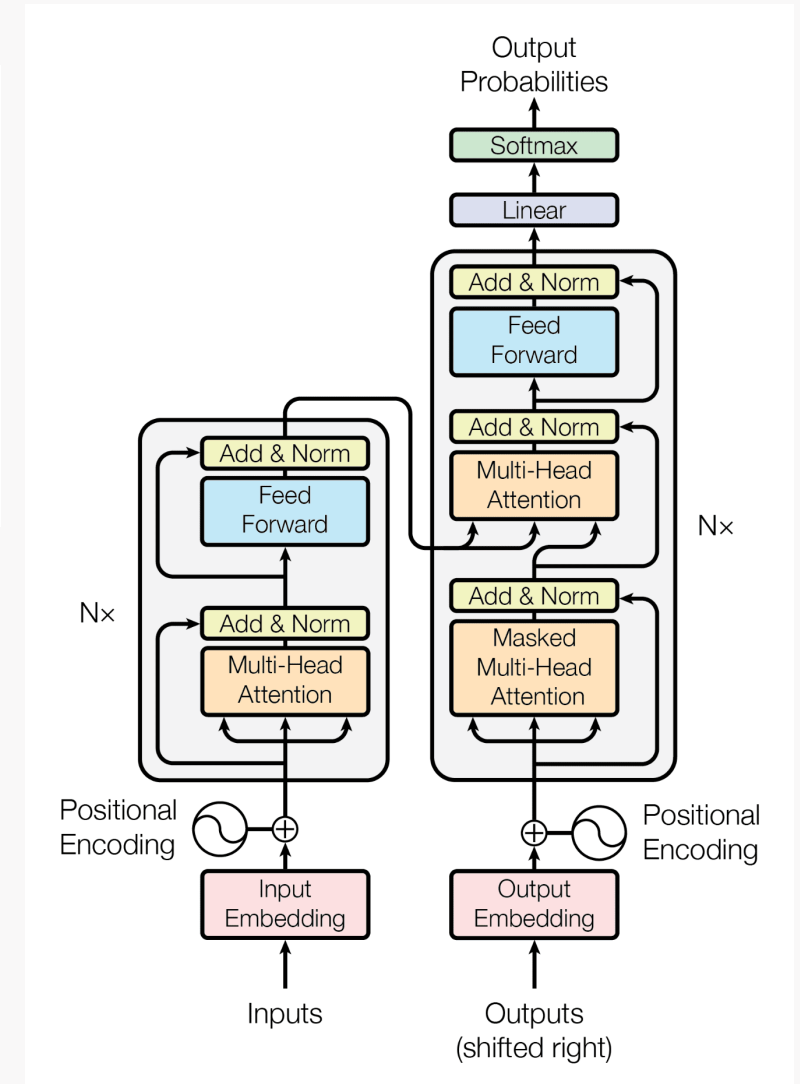
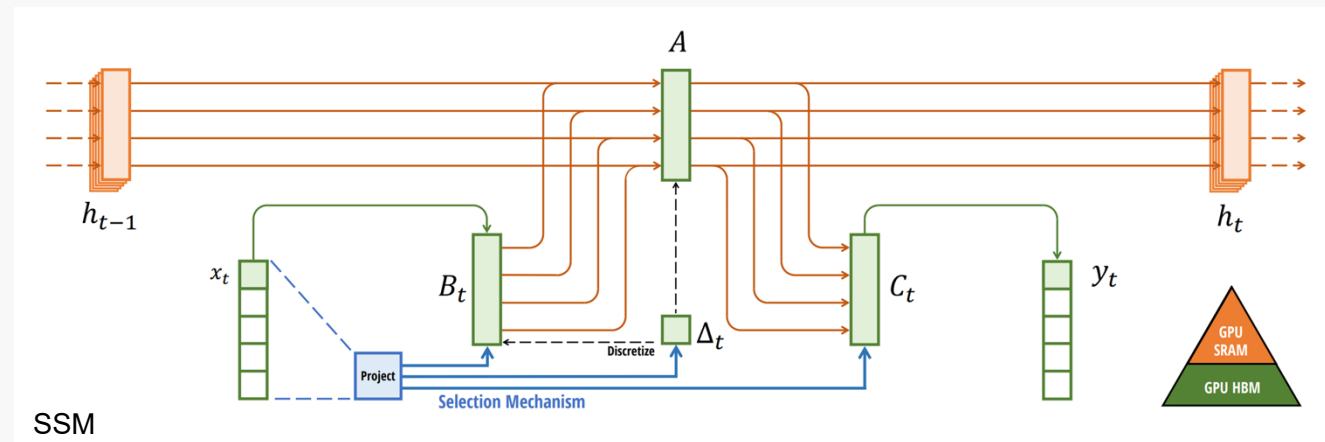
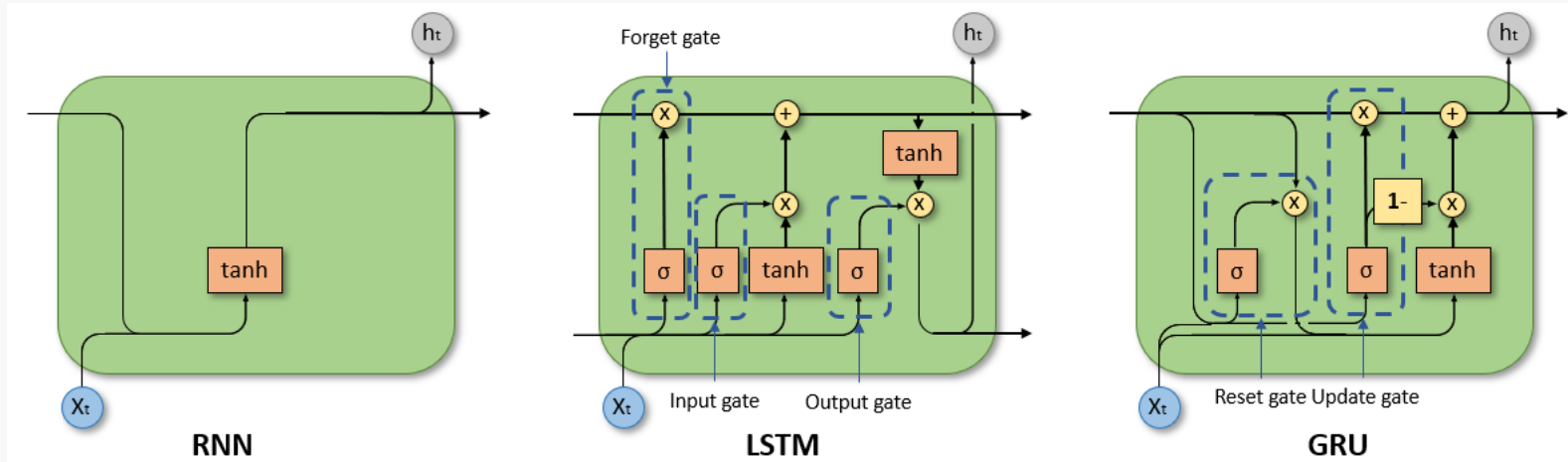
Modeling and predicting complex dynamics have motivated the development of a wide range of machine learning (ML) models.

ML models that predict/generate complex dynamics

ML models that predict/generate complex dynamics



ML models that predict/generate complex dynamics



A Key Question

Do these models rely on shared principles to predict complex dynamics?

Generlized Synchronization vs Hopping



Recurrent Models:

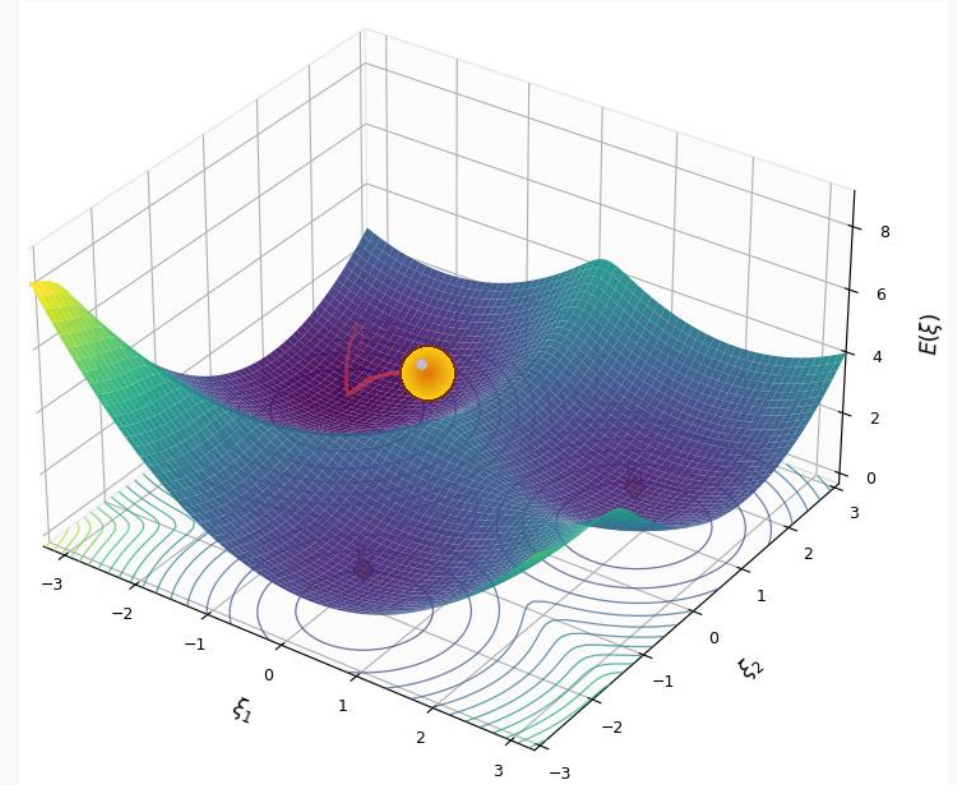
The recurrent models can be considered as dynamical systems that is one-way coupled to the input dynamical system.

Generalized Synchronization vs Hopping



Recurrent Models:

The recurrent models can be considered as **dynamical systems** that is **one-way coupled** to the input dynamical system.



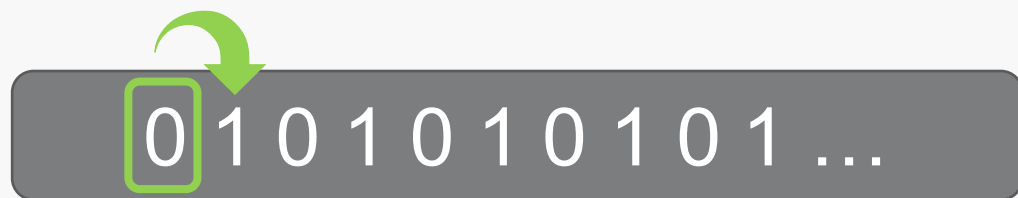
Attention-based Models:

The generation of the successive token can be viewed as a **hopping process on an evolving energy landscape**.

I. How Do Recurrent Models Predict Complex Dynamics?

Predicting complex dynamics needs access to the history of the input (delayed embedding). Recurrent models hold memories of the input history via the “generalized synchronization”.

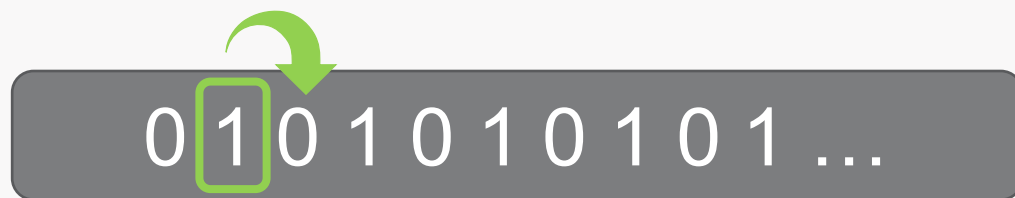
Why do we need delayed embeddings?



For simple dynamics, the prediction is purely a function of the previous input, and thus a feedforward model suffices!

Feedforward Model
 $o(t) = f(x(t-1))$

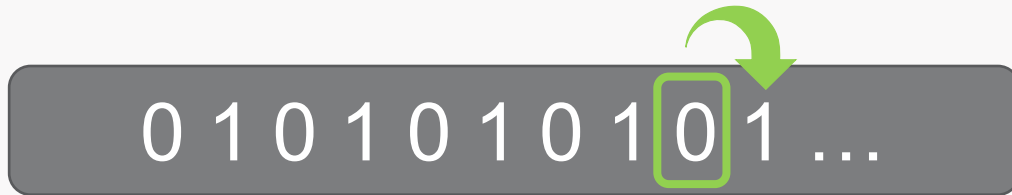
Why do we need delayed embeddings?



For simple dynamics, the prediction is purely a function of the previous input, and thus a feedforward model suffices!

Feedforward Model
 $o(t) = f(x(t-1))$

Why do we need delayed embeddings?



For simple dynamics, the prediction is purely a function of the previous input, and thus a feedforward model suffices!

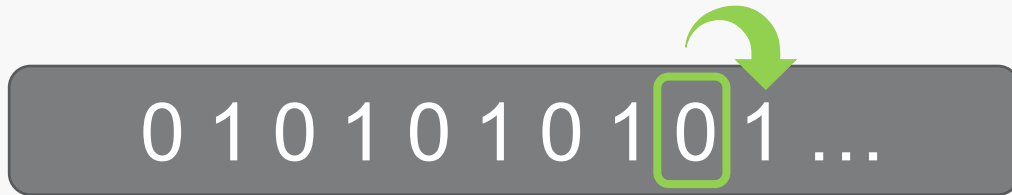
Feedforward Model
 $o(t) = f(x(t-1))$



For complex dynamics, the prediction is no longer a function of the previous input, but the input history! Thus, prediction needs the delay embedding: $o(t) = h(x(t-1), x(t-1), \dots)$.

Recurrent Model:
 $r(t) = f(r(t-1), x(t-1))$
 $o(t) = g(r(t))$
Generalized synchronization:
 $o(t) = h(x(t-1), x(t-1), \dots)$

Why do we need delayed embeddings?



For simple dynamics, the prediction is purely a function of the previous input, and thus a feedforward model suffices!

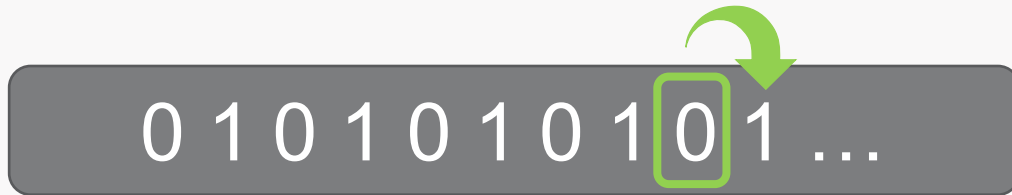
Feedforward Model
 $o(t) = f(x(t-1))$



For complex dynamics, the prediction is no longer a function of the previous input, but the input history! Thus, prediction needs the delay embedding: $o(t) = h(x(t-1), x(t-1), \dots)$.

Recurrent Model:
 $r(t) = f(r(t-1), x(t-1))$
 $o(t) = g(r(t))$
Generalized synchronization:
 $o(t) = h(x(t-1), x(t-1), \dots)$

Why do we need delayed embeddings?



For simple dynamics, the prediction is purely a function of the previous input, and thus a feedforward model suffices!

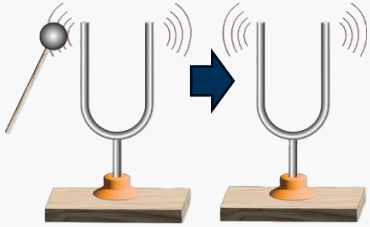
Feedforward Model:
 $o(t) = f(x(t-1))$



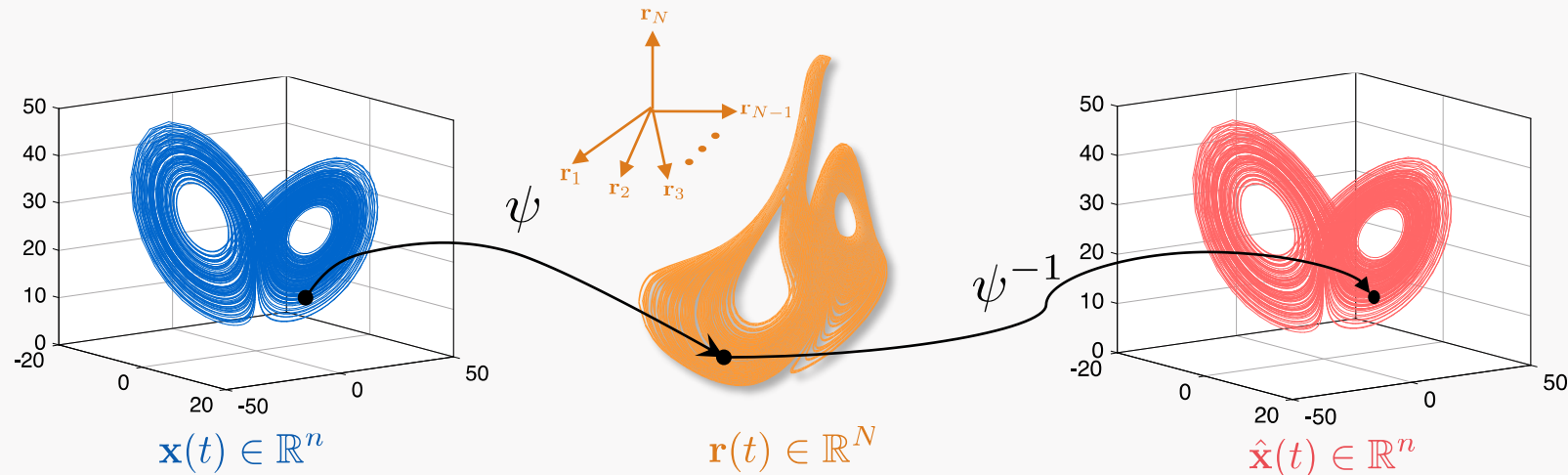
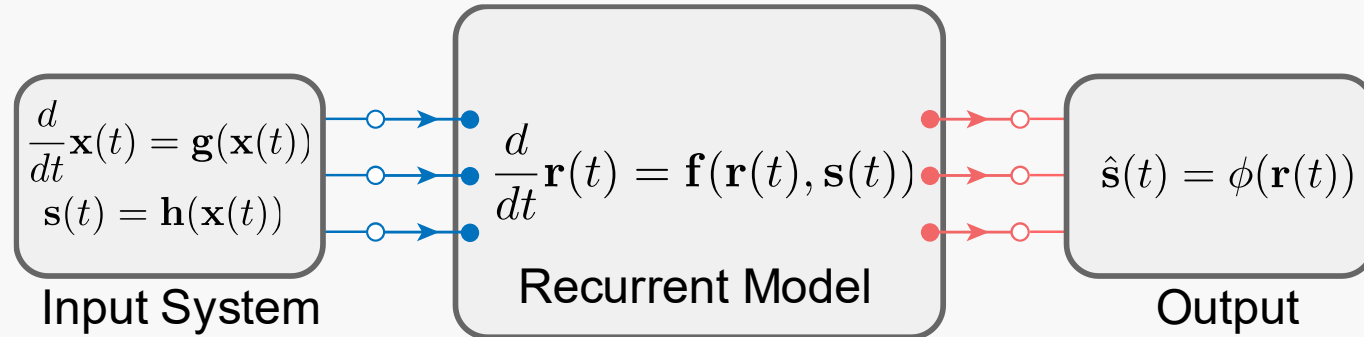
For complex dynamics, the prediction is no longer a function of the previous input, but the input history! Thus, prediction needs the delay embedding: $o(t) = h(x(t-1), x(t-1), \dots)$.

Recurrent Model:
 $r(t) = f(r(t-1), x(t-1))$
 $o(t) = g(r(t))$
Generalized synchronization:
 $o(t) = h(x(t-1), x(t-1), \dots)$

Example: Learning to predict the Lorenz system

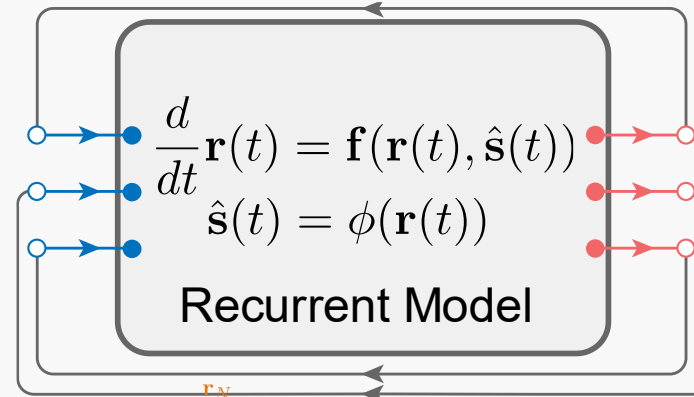


To Learn is to Synchronize

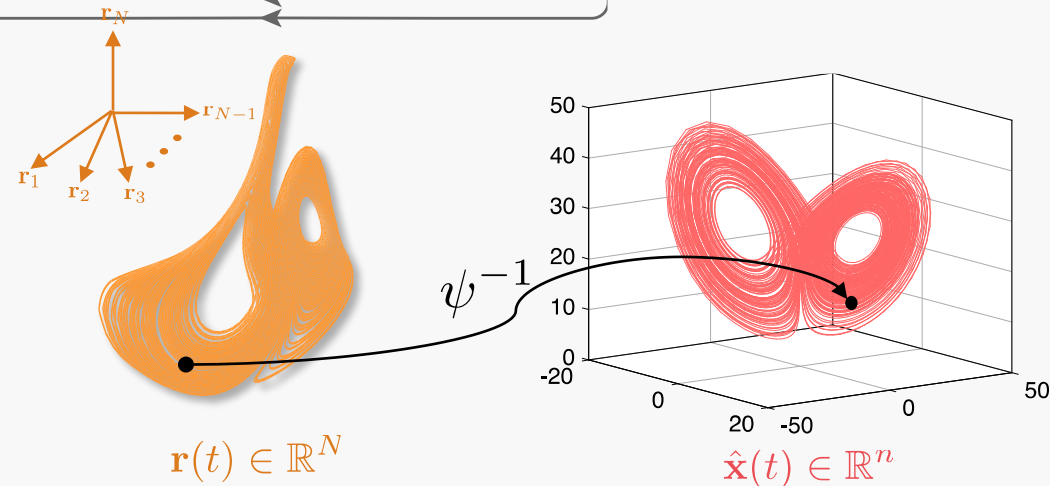


Example: Learning to predict the Lorenz system

To Predict is to Generate in Close-loop

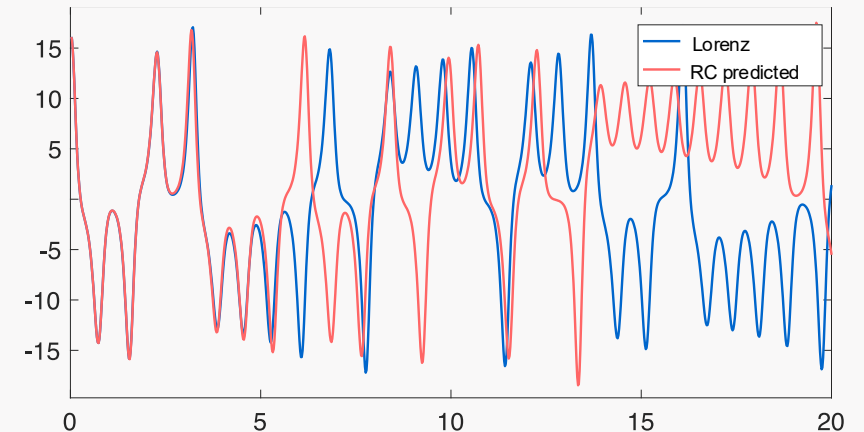
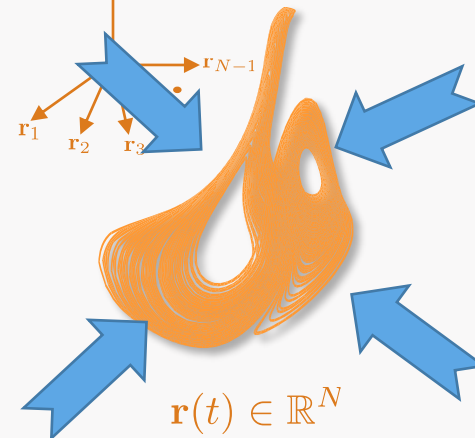
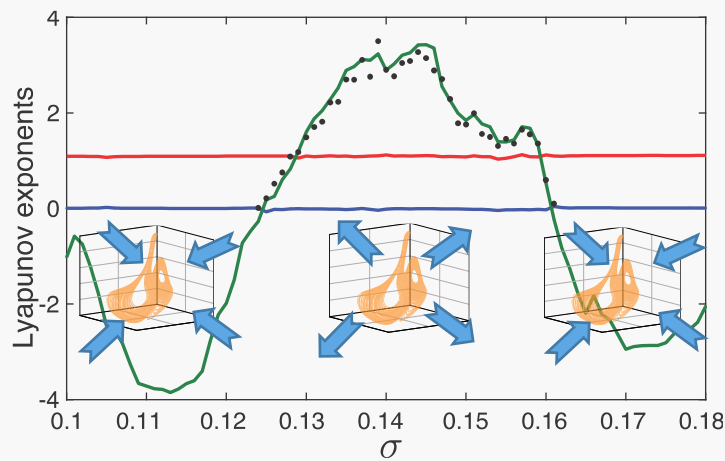
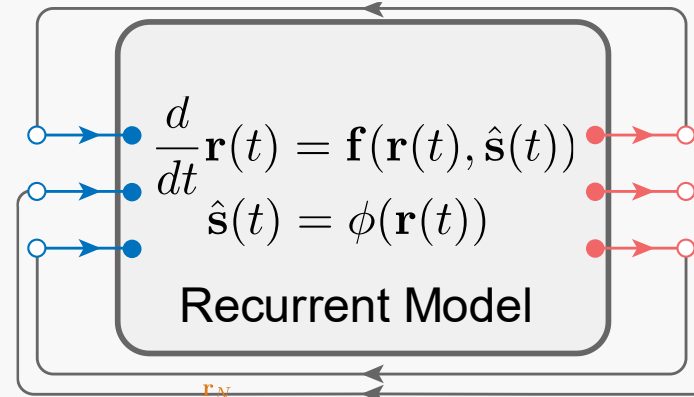


The recurrent model then predicts the future by autonomously evolving on the **embedded attractor**.



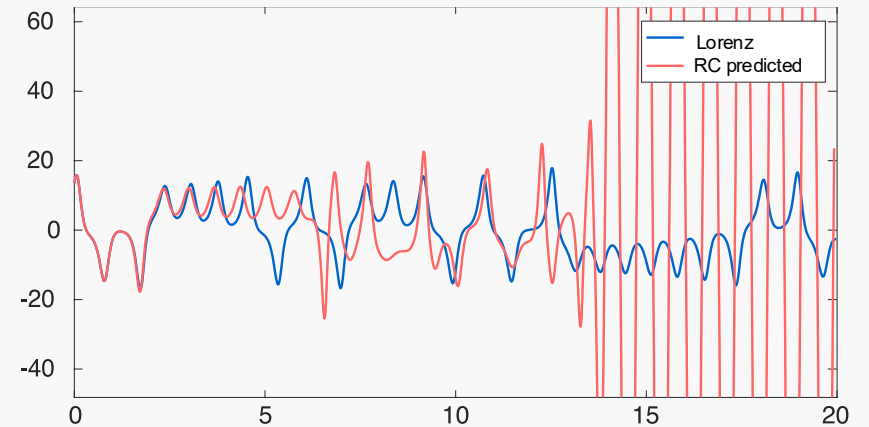
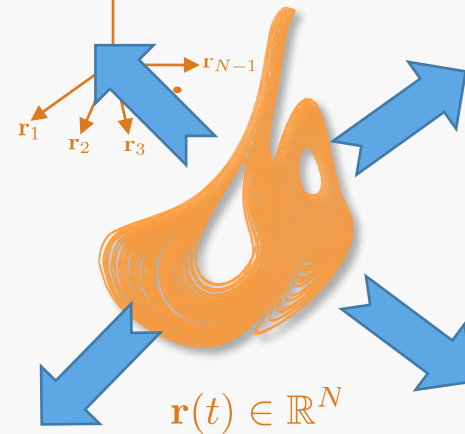
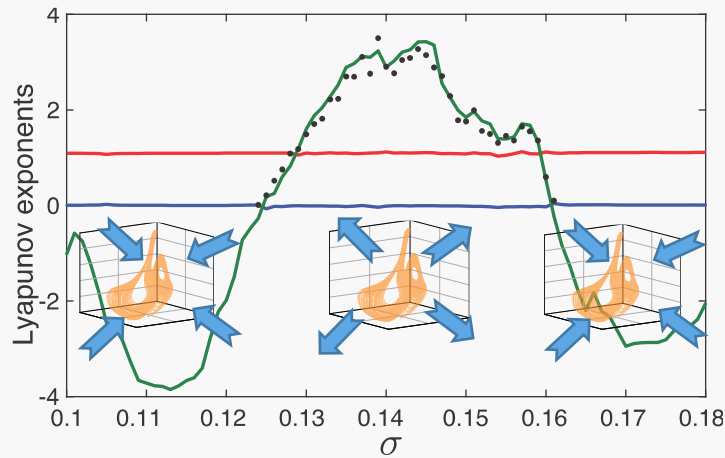
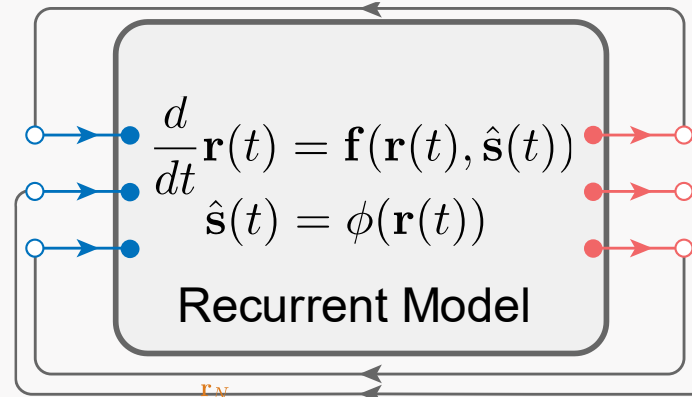
Example: Learning to predict the Lorenz system

To Predict is to Generate in Close-loop



The transverse stability is determined by the sign of the **spurious Lyapunov exponent**, in the post-learning network. Such a Lyapunov exponent also predicts the escaping rate of the close-loop network.

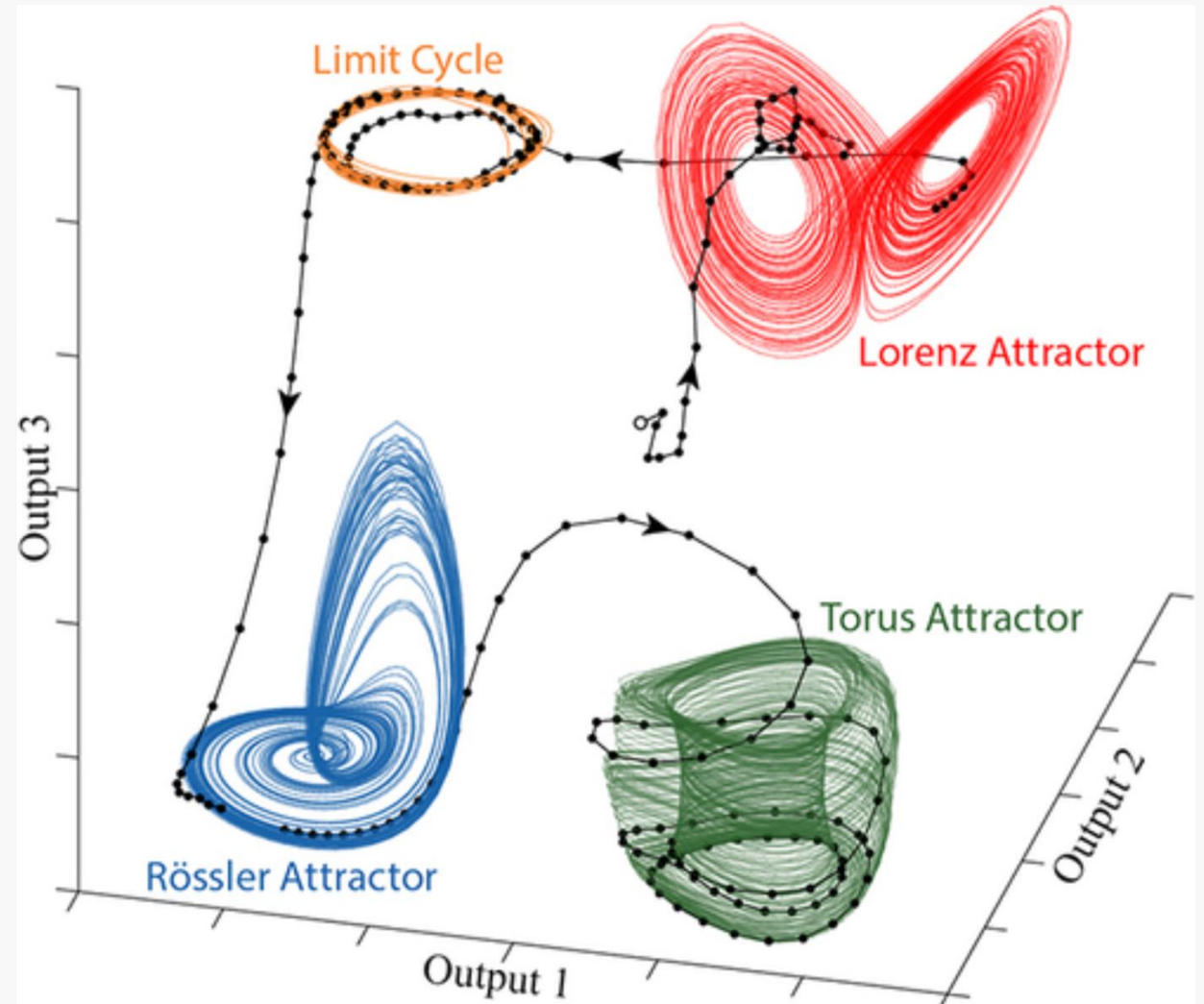
To Predict is to Generate in Close-loop



Going Beyond Predictions

Multi-task Learning:

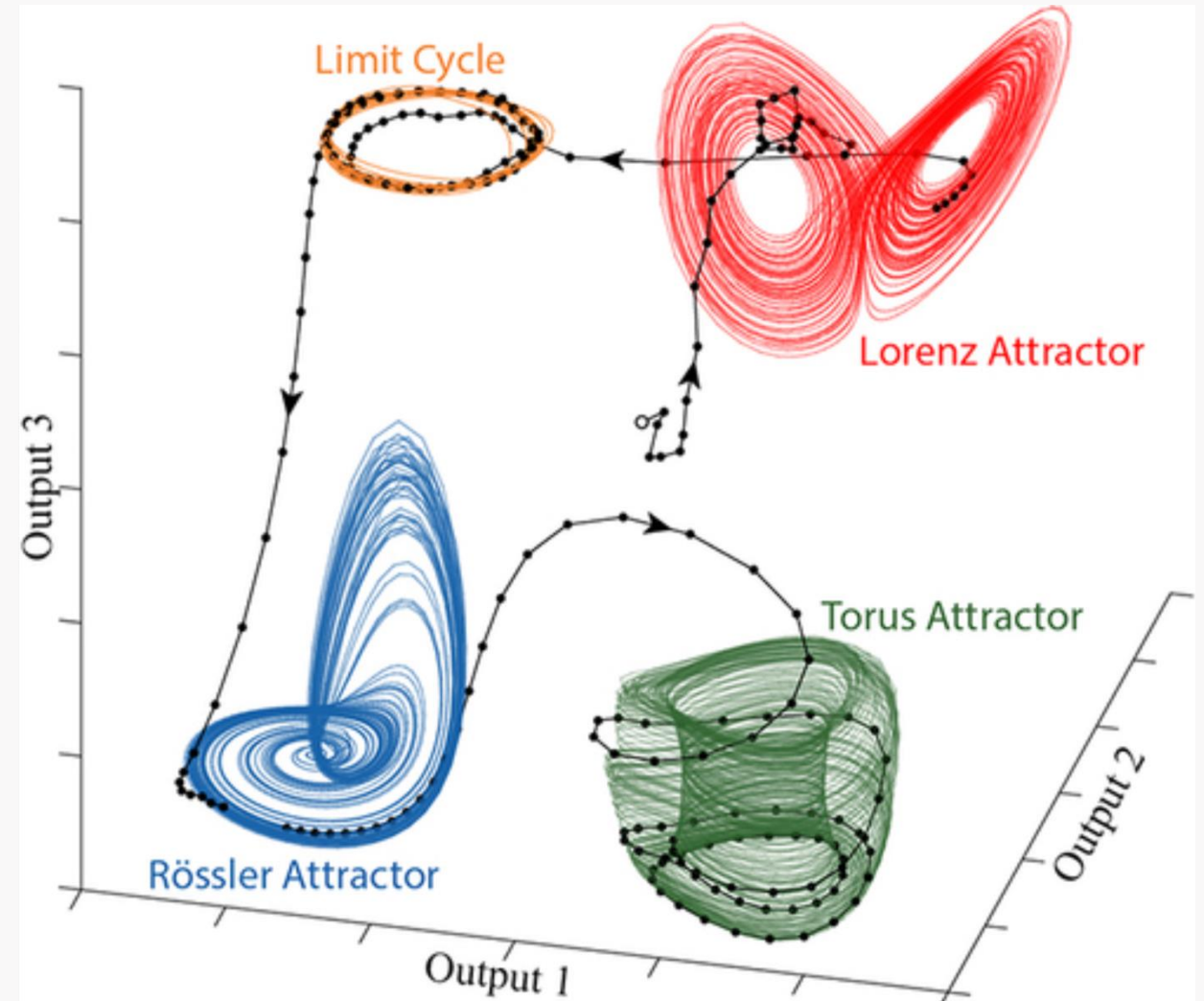
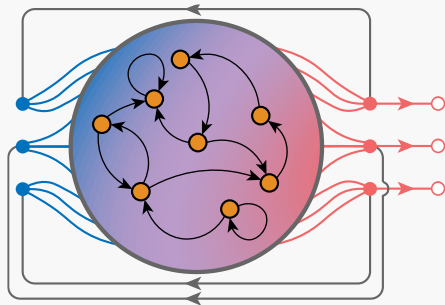
A single recurrent neural network can be trained to imitate the behavior of many distinctive dynamical attractors, exhibiting enforced and autonomous task switching



Going Beyond Predictions

Multi-task Learning:

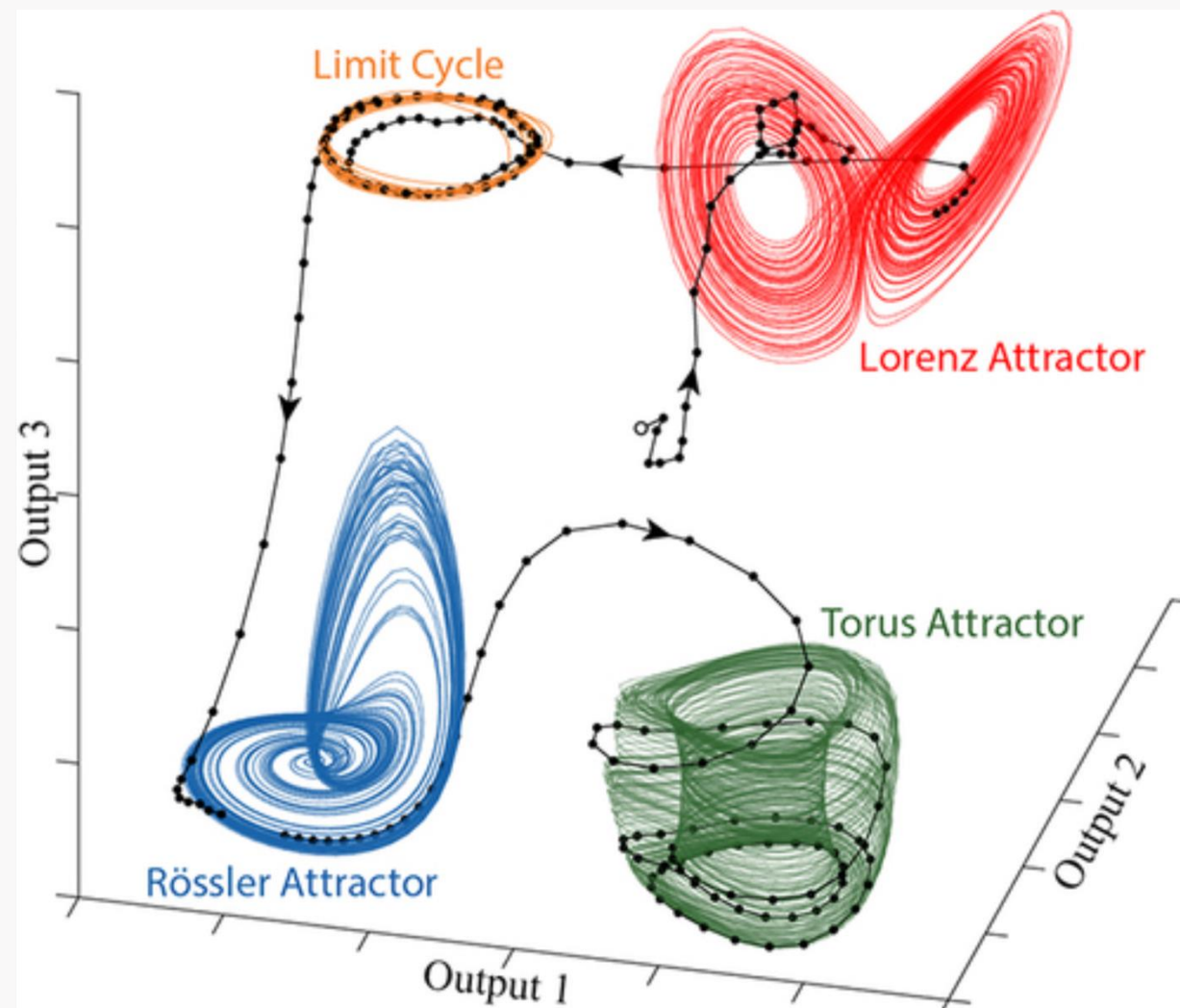
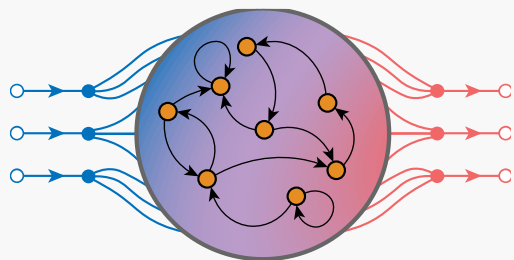
A single recurrent neural network can be trained to imitate the behavior of many distinctive dynamical attractors, exhibiting enforced and autonomous task switching



Going Beyond Predictions

Multi-task Learning:

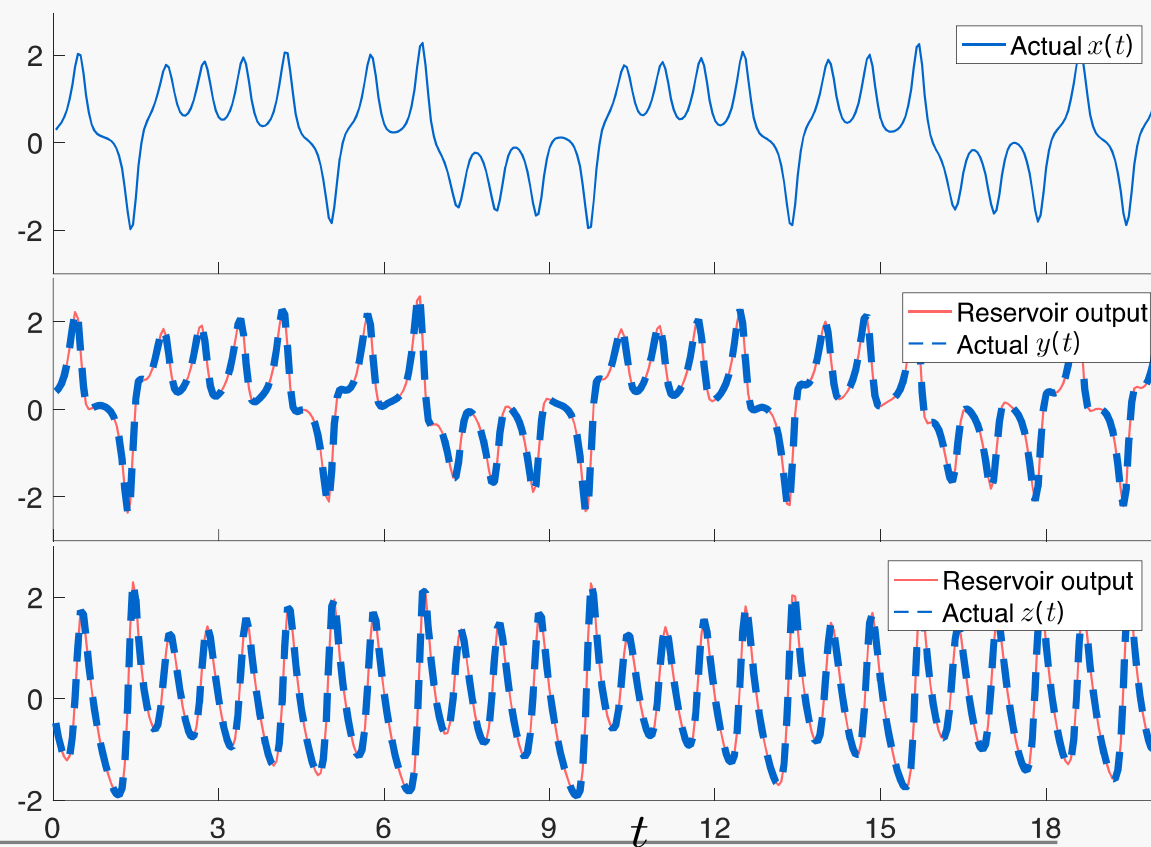
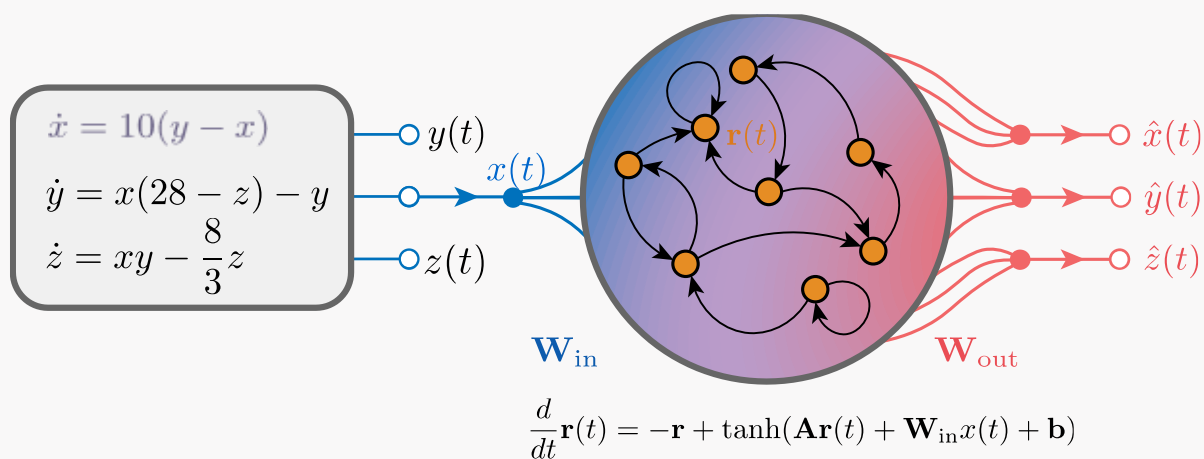
A single recurrent neural network can be trained to imitate the behavior of many distinctive dynamical attractors, exhibiting enforced and autonomous task switching



Going Beyond Predictions

Latent Variable Inference:

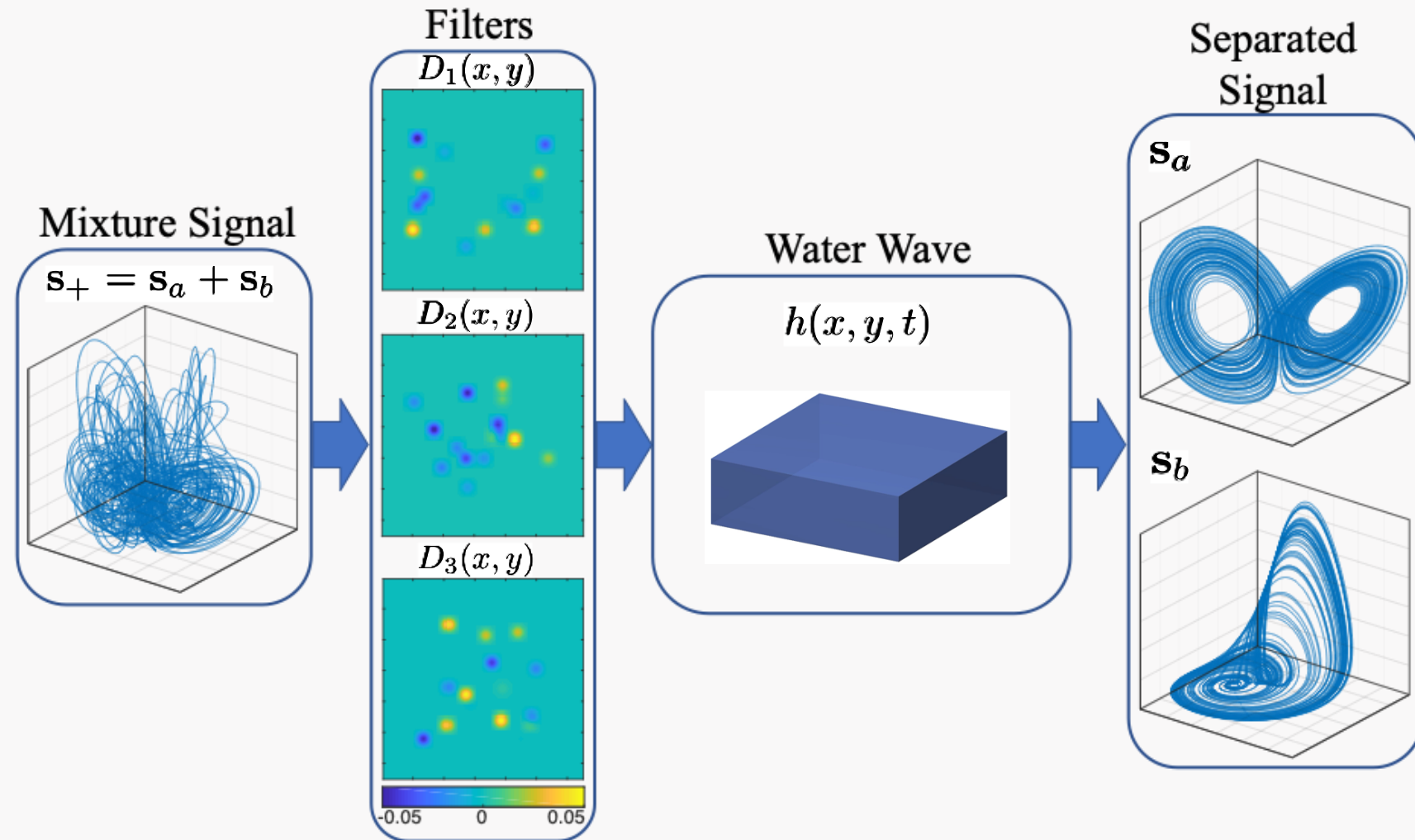
The recurrent models can use the delayed embedding obtained from the generalized synchronization to infer the **unseen latent variables** of the input system.



Going Beyond Predictions

Source Separation:

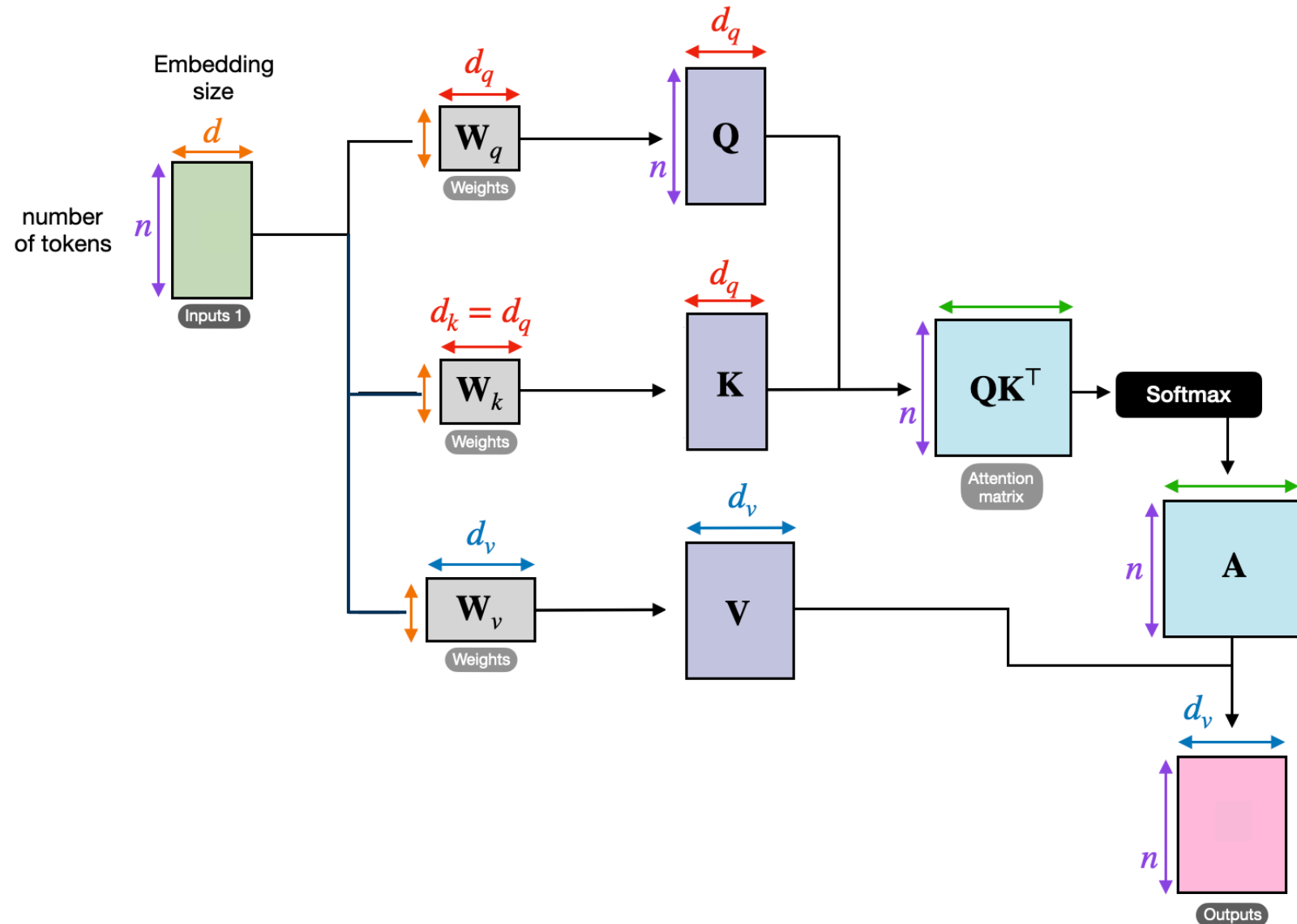
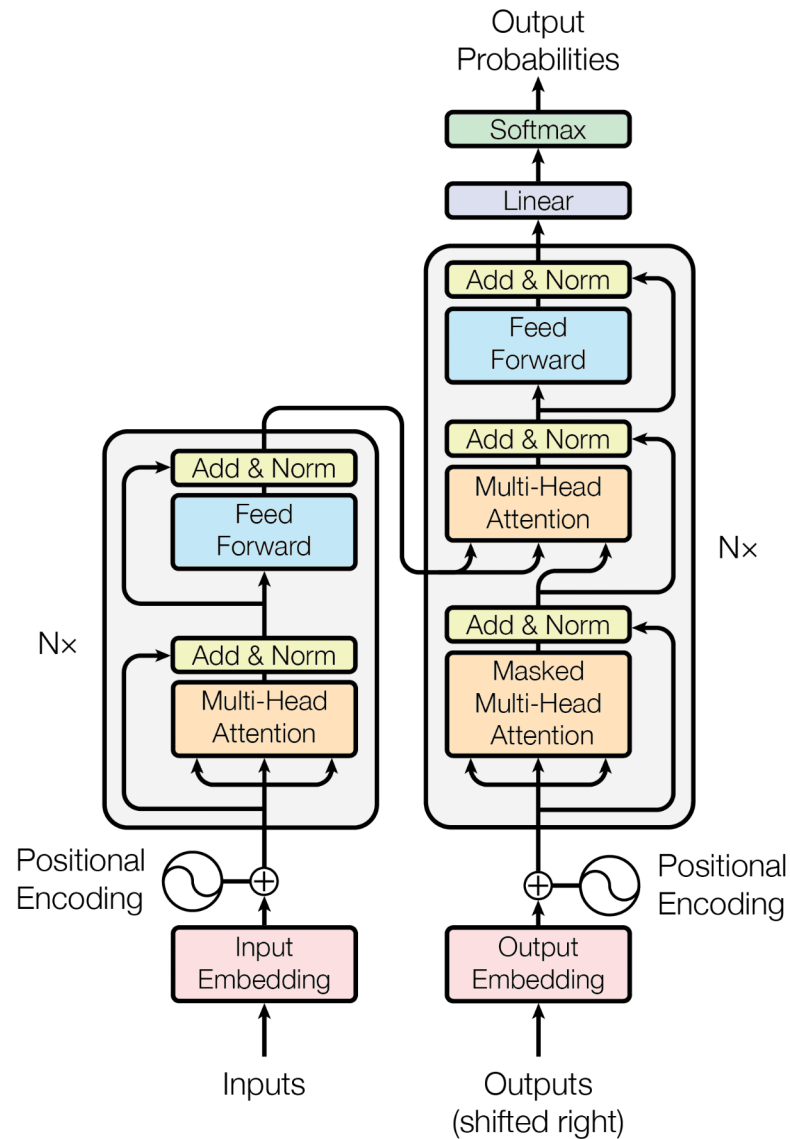
Recurrent models can, via the generalized synchronization, infer the separated dynamical signals of different sources. A tank of water can also do that.



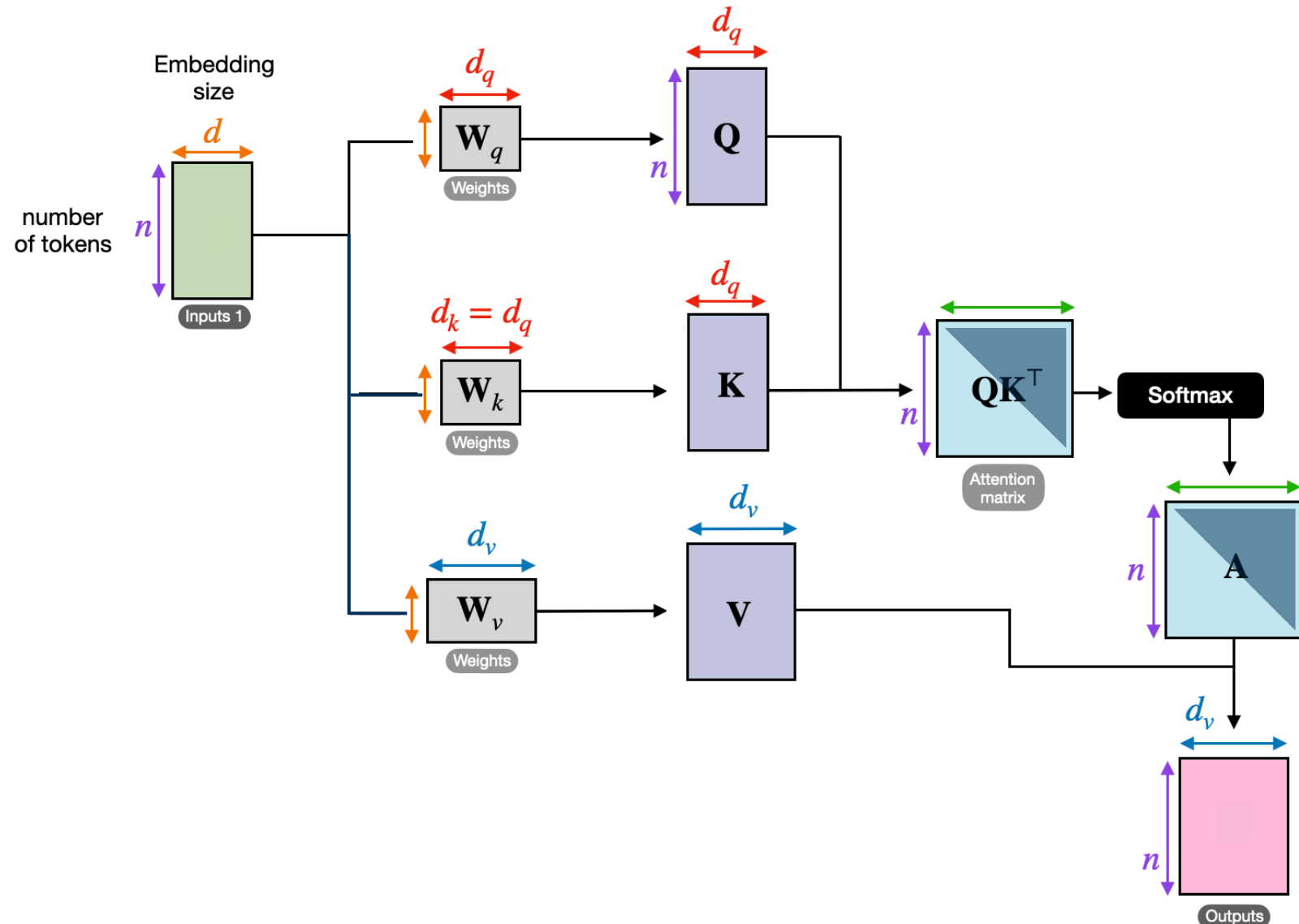
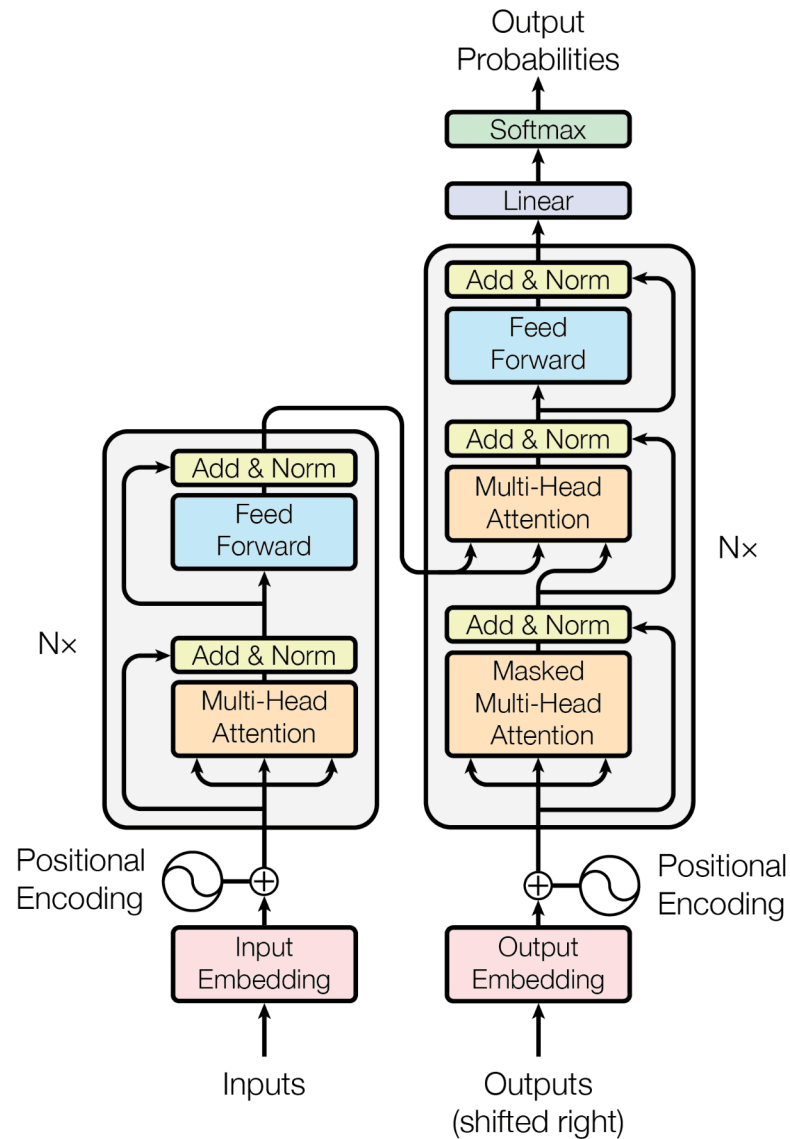
II. Ho do Attention Models Predict the Next Token?

In attention-based models, an evolving energy landscape retains the memory of previous tokens and directs how the next token is generated through a hopping mechanism.

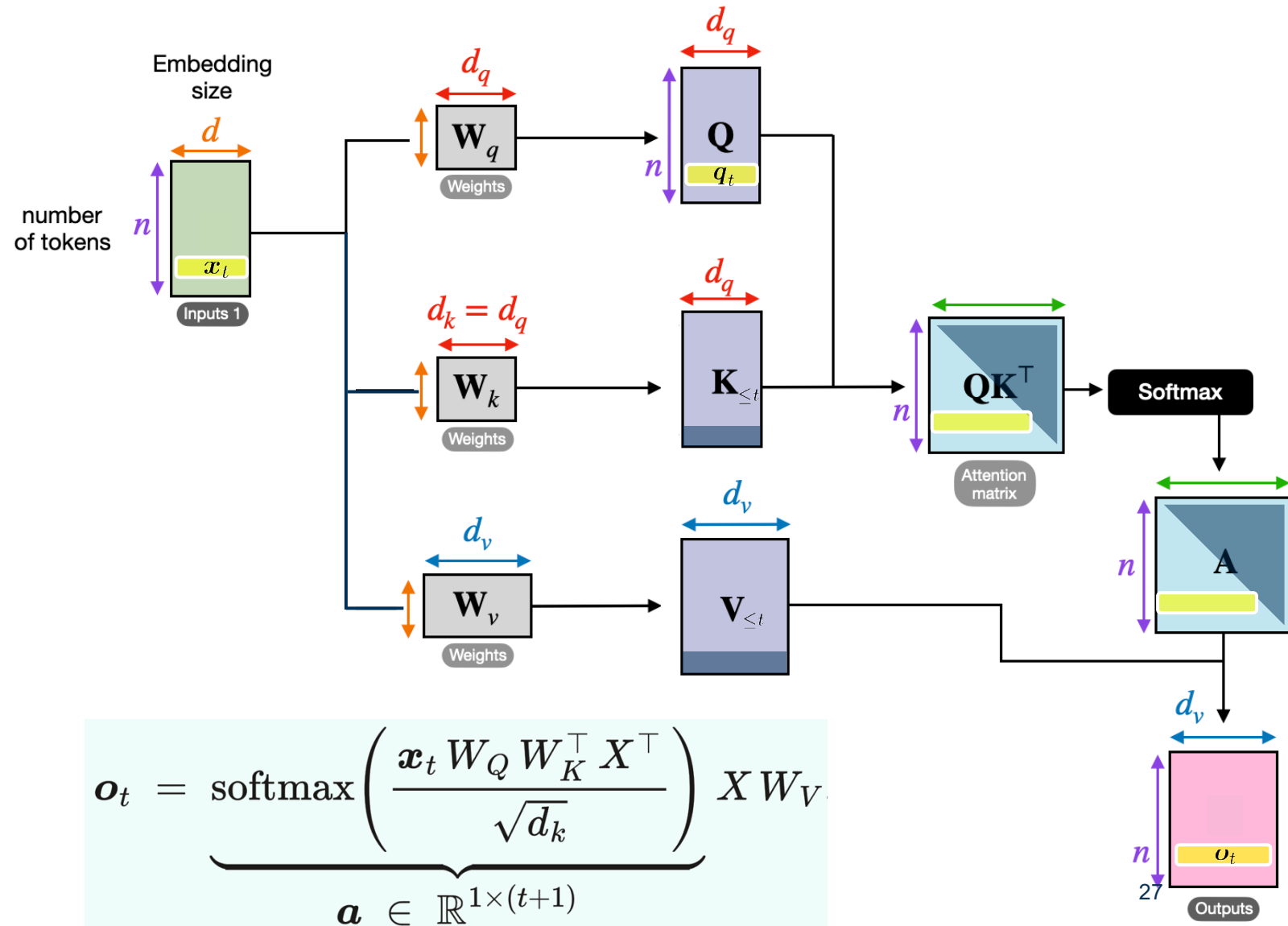
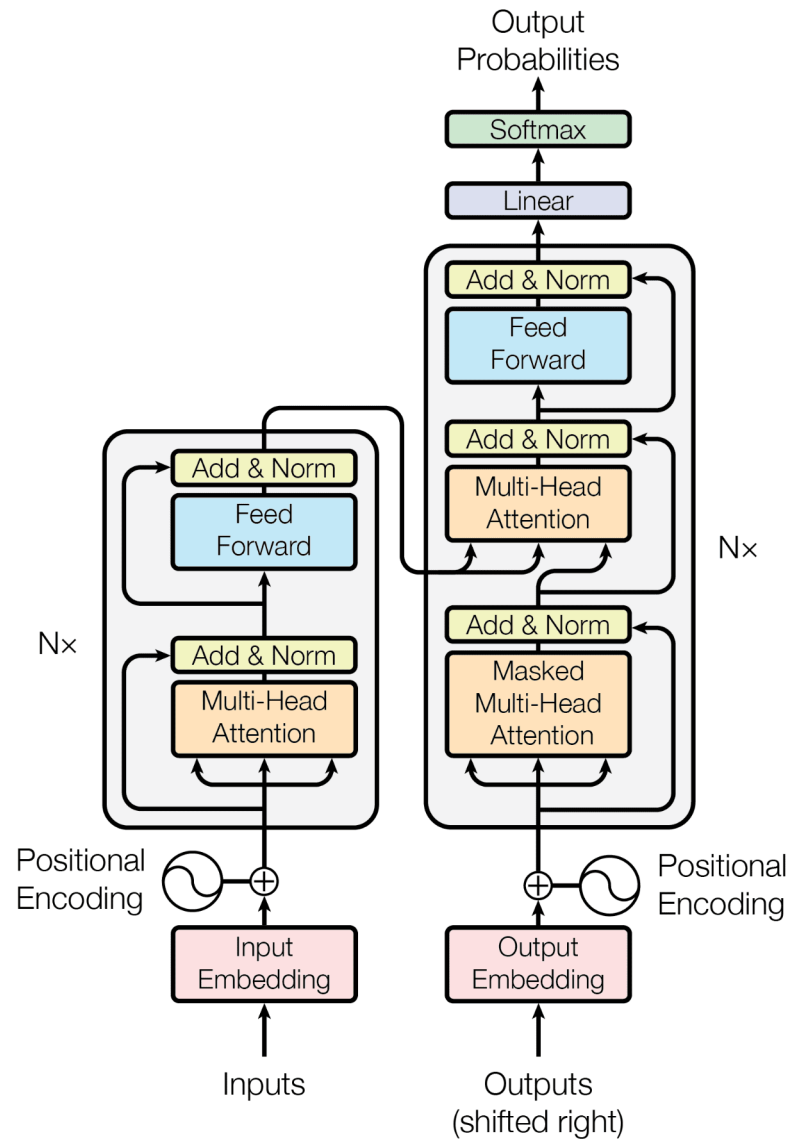
Transformer and the Attention Block



Transformer and the Attention Block



Transformer and the Attention Block



Attention Hops

$$\mathbf{o}_t = \text{softmax} \left(\underbrace{\frac{\mathbf{x}_t \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{X}^\top}{\sqrt{d_k}}}_{\mathbf{a}_t \in \mathbb{R}^{1 \times (t+1)}} \right) \mathbf{X} \mathbf{W}_V$$

Attention Hops

$$\mathbf{o}_t = \underbrace{\text{softmax} \left(\frac{\mathbf{x}_t \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{X}^\top}{\sqrt{d_k}} \right)}_{\mathbf{a}_t \in \mathbb{R}^{1 \times (t+1)}} \mathbf{X} \mathbf{W}_V$$

$$\mathbf{Y} := \mathbf{X} \mathbf{W}_K \mathbf{W}_Q^\top \quad \mathbf{Y} = \begin{bmatrix} \mathbf{y}_0 \\ \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_t \end{bmatrix} \in \mathbb{R}^{(t+1) \times d}$$

Attention Hops

$$\mathbf{o}_t = \underbrace{\text{softmax} \left(\frac{\mathbf{x}_t \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{X}^\top}{\sqrt{d_k}} \right)}_{\mathbf{a}_t \in \mathbb{R}^{1 \times (t+1)}} \mathbf{X} \mathbf{W}_V$$

$$\mathbf{Y} := \mathbf{X} \mathbf{W}_K \mathbf{W}_Q^\top \quad \mathbf{Y} = \begin{bmatrix} \mathbf{y}_0 \\ \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_t \end{bmatrix} \in \mathbb{R}^{(t+1) \times d}$$

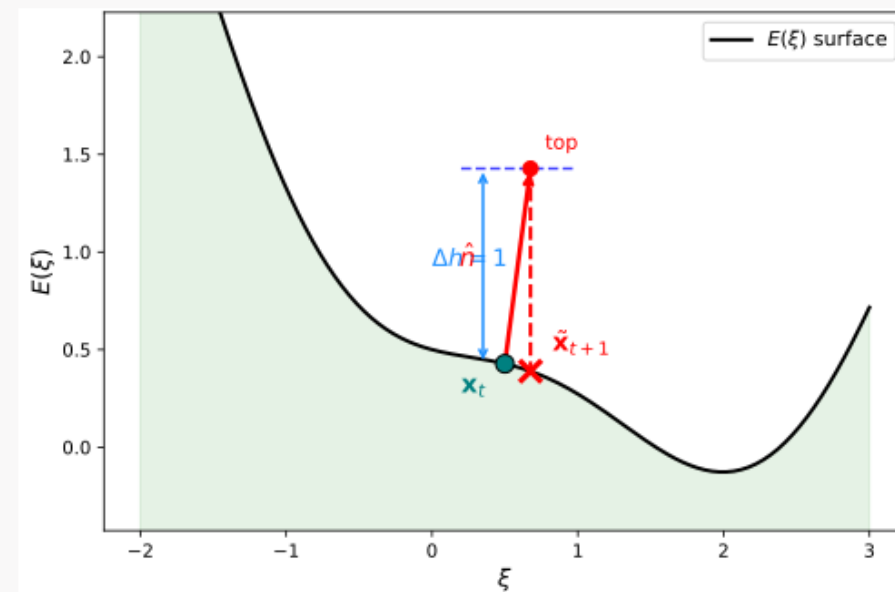
$$E_{\mathbf{Y}}(\mathbf{x}) = -\log \sum_{\tau=0}^t \underbrace{\exp \left(\frac{\|\mathbf{y}_\tau\|^2}{2\sqrt{d_k}} \right)}_{\text{mode strength}} \exp \left(-\frac{\|\mathbf{x} - \mathbf{y}_\tau\|^2}{2\sqrt{d_k}} \right) + C.$$

Attention Hops

$$\mathbf{o}_t = \underbrace{\text{softmax} \left(\frac{\mathbf{x}_t \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{X}^\top}{\sqrt{d_k}} \right)}_{\mathbf{a}_t \in \mathbb{R}^{1 \times (t+1)}} \mathbf{X} \mathbf{W}_V$$

$$\mathbf{Y} := \mathbf{X} \mathbf{W}_K \mathbf{W}_Q^\top \quad \mathbf{Y} = \begin{bmatrix} \mathbf{y}_0 \\ \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_t \end{bmatrix} \in \mathbb{R}^{(t+1) \times d}$$

$$E_{\mathbf{Y}}(\mathbf{x}) = -\log \underbrace{\sum_{\tau=0}^t \exp \left(\frac{\|\mathbf{y}_\tau\|^2}{2\sqrt{d_k}} \right)}_{\text{mode strength}} \exp \left(-\frac{\|\mathbf{x} - \mathbf{y}_\tau\|^2}{2\sqrt{d_k}} \right) + C.$$



Attention Hops

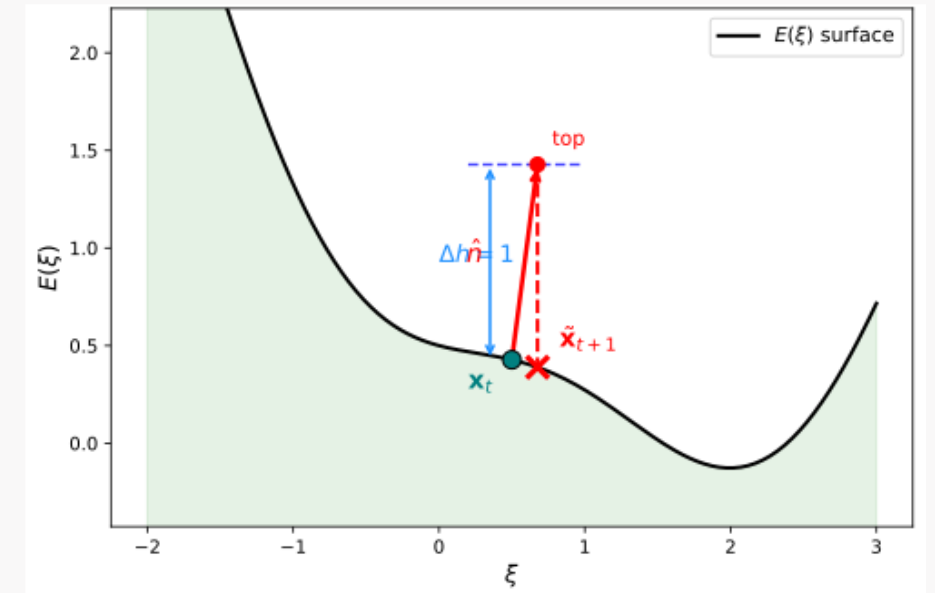
$$\mathbf{o}_t = \underbrace{\text{softmax} \left(\frac{\mathbf{x}_t \mathbf{W}_Q \mathbf{W}_K^\top \mathbf{X}^\top}{\sqrt{d_k}} \right)}_{\mathbf{a}_t \in \mathbb{R}^{1 \times (t+1)}} \mathbf{X} \mathbf{W}_V$$

$$\mathbf{Y} := \mathbf{X} \mathbf{W}_K \mathbf{W}_Q^\top \quad \mathbf{Y} = \begin{bmatrix} \mathbf{y}_0 \\ \mathbf{y}_1 \\ \vdots \\ \mathbf{y}_t \end{bmatrix} \in \mathbb{R}^{(t+1) \times d}$$

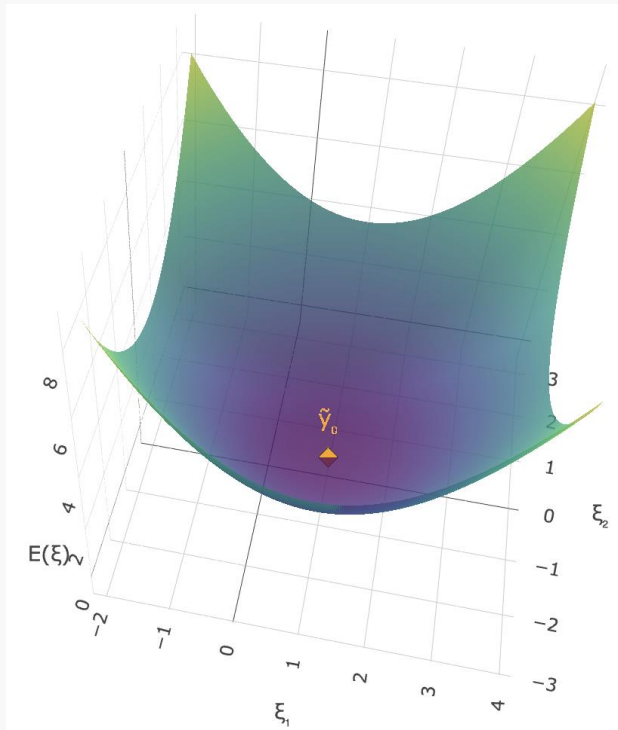
$$E_{\mathbf{Y}}(\mathbf{x}) = -\log \underbrace{\sum_{\tau=0}^t \exp \left(\frac{\|\mathbf{y}_\tau\|^2}{2\sqrt{d_k}} \right)}_{\text{mode strength}} \exp \left(-\frac{\|\mathbf{x} - \mathbf{y}_\tau\|^2}{2\sqrt{d_k}} \right) + C.$$

$$\mathbf{o}_t = \text{softmax} \left(\underbrace{\frac{\overbrace{\mathbf{x}_t}^{\text{hopping location}} \underbrace{\mathbf{Y}^\top}_{\text{modes}}}{\sqrt{d_k}}}_{\tilde{\mathbf{x}}_{t+1} \text{ landing location}} \right) \mathbf{W}_{\text{read}}$$

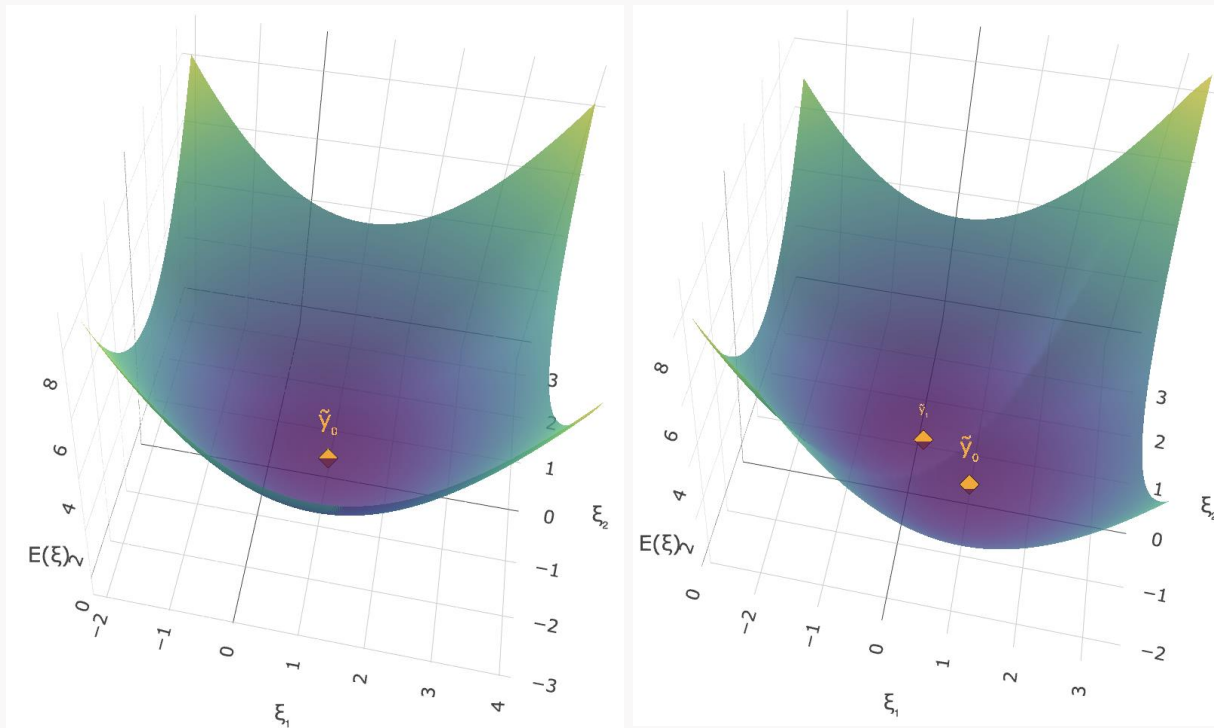
$$\mathbf{W}_{\text{read}} := \left(\mathbf{W}_K \mathbf{W}_Q^\top \right)^+ \mathbf{W}_V$$



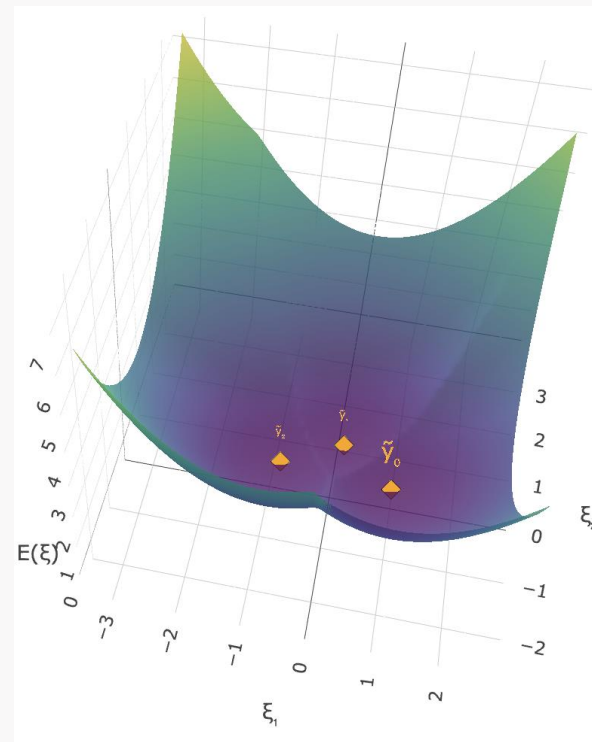
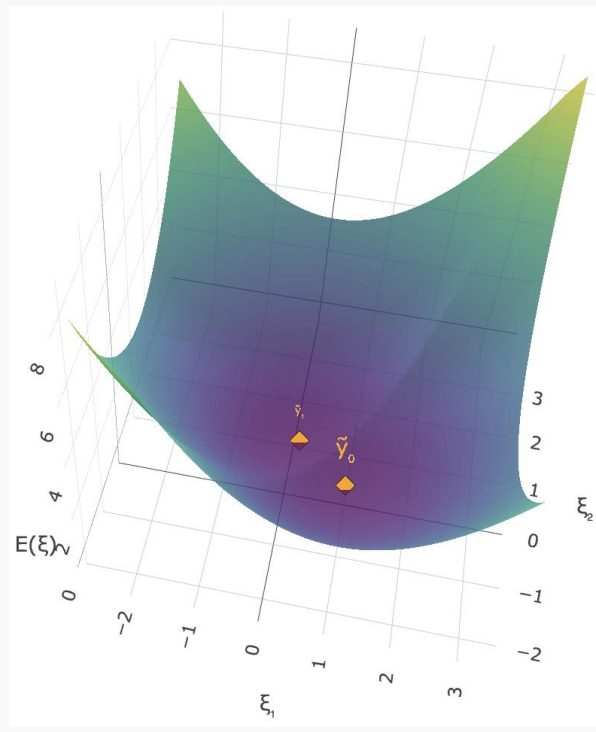
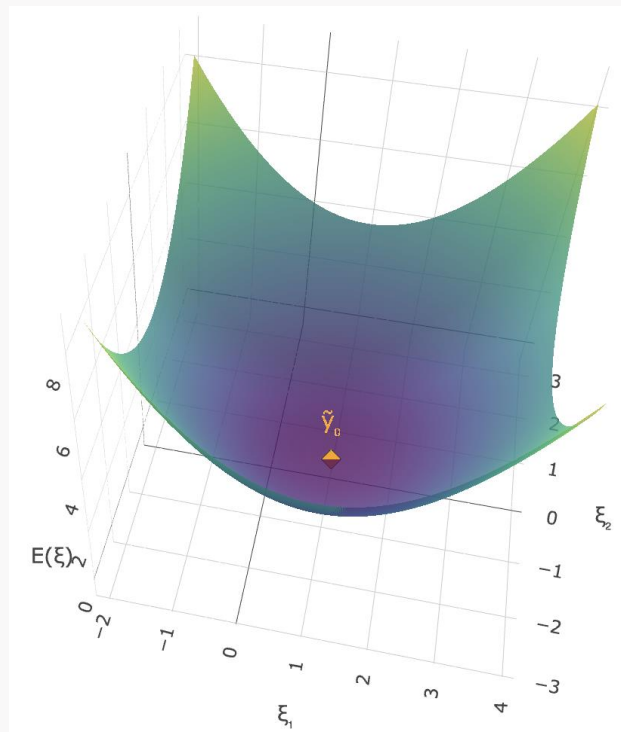
Evolving Energy Landscape:



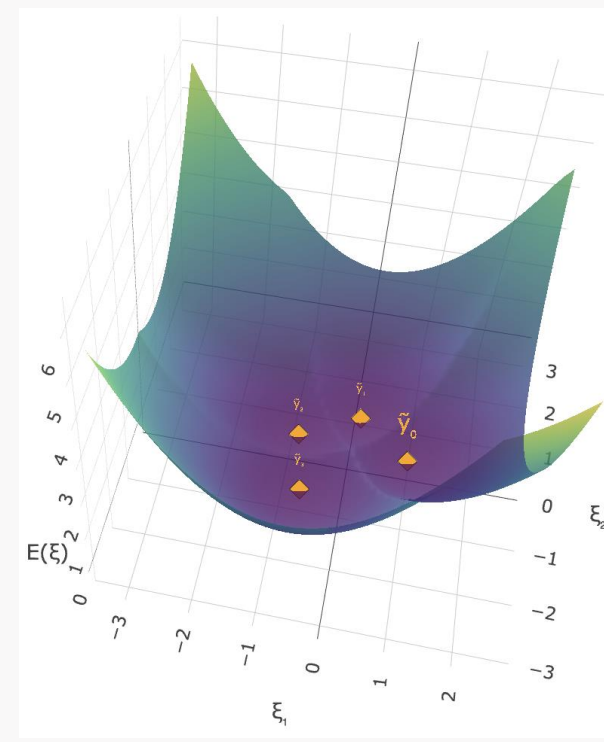
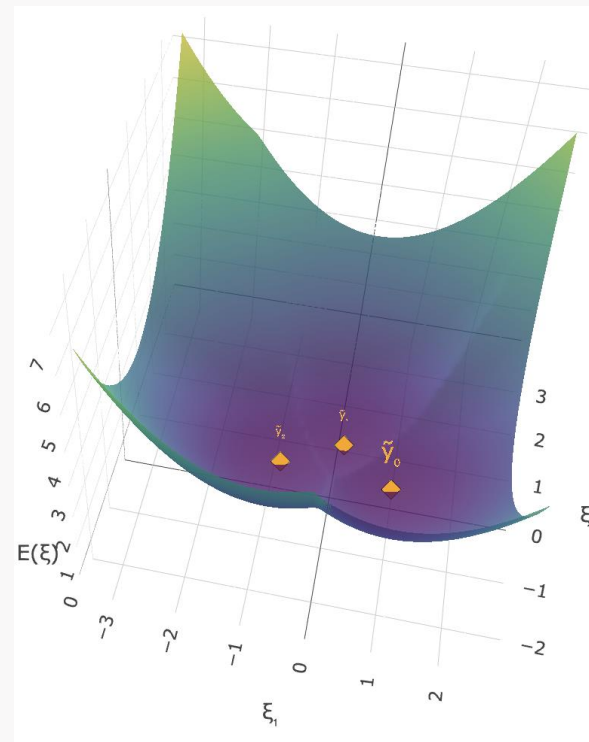
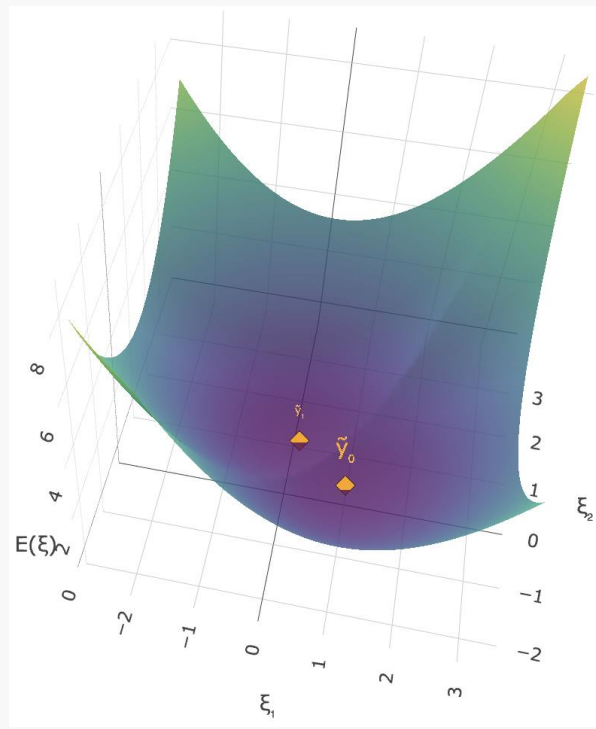
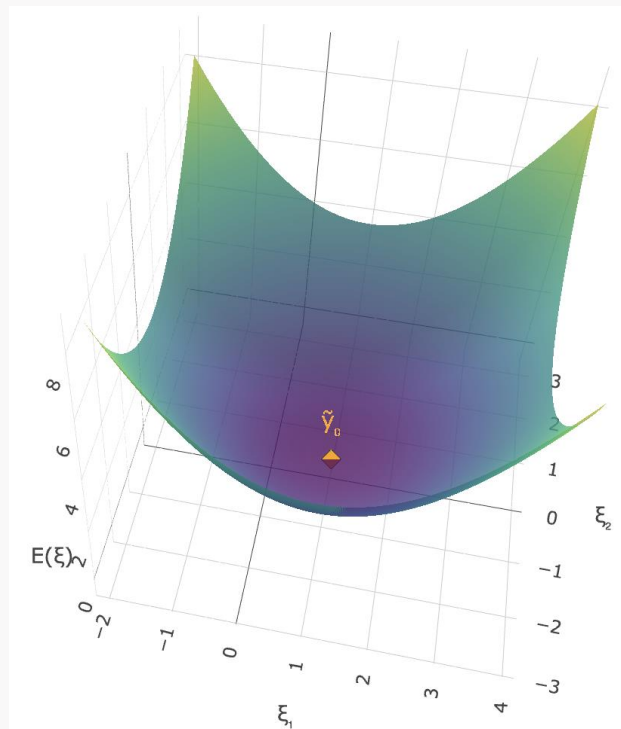
Evolving Energy Landscape:



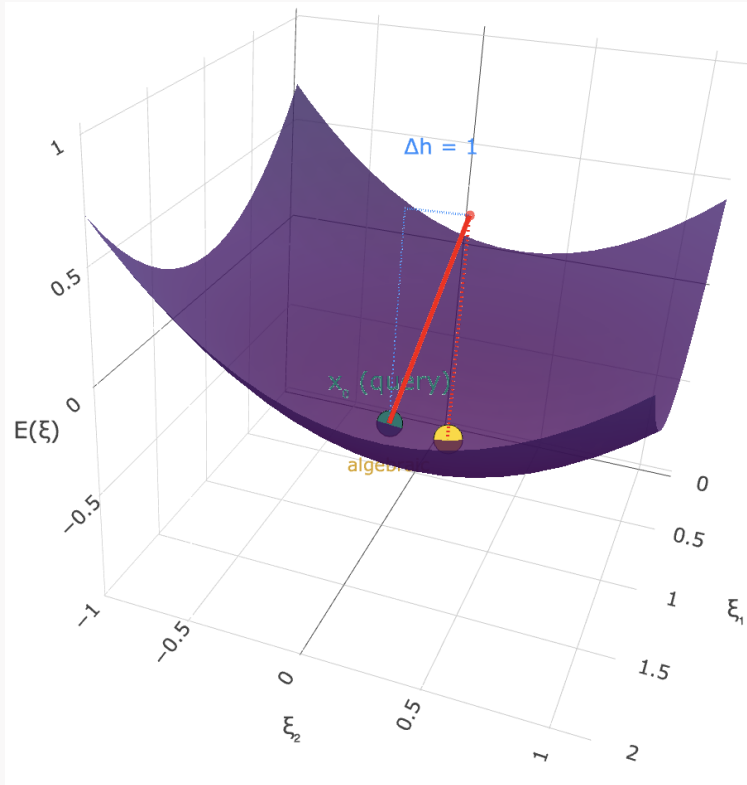
Evolving Energy Landscape:



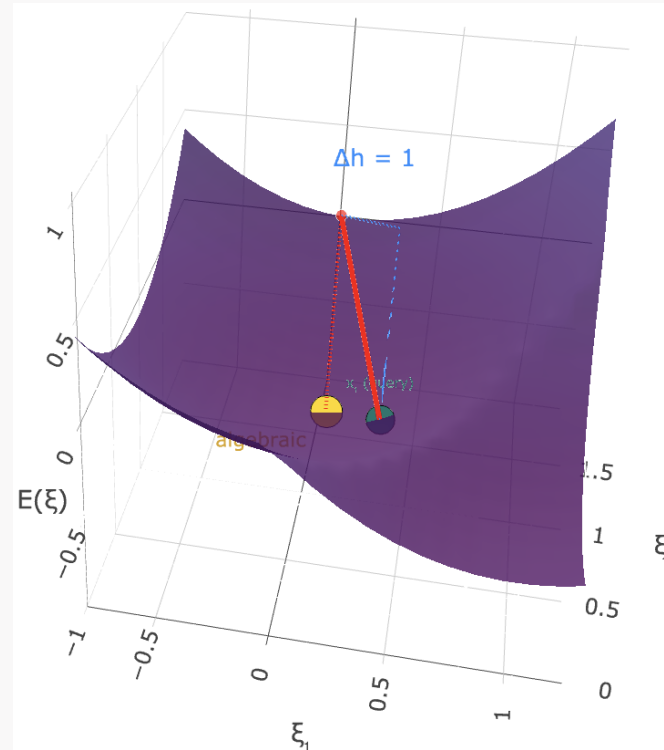
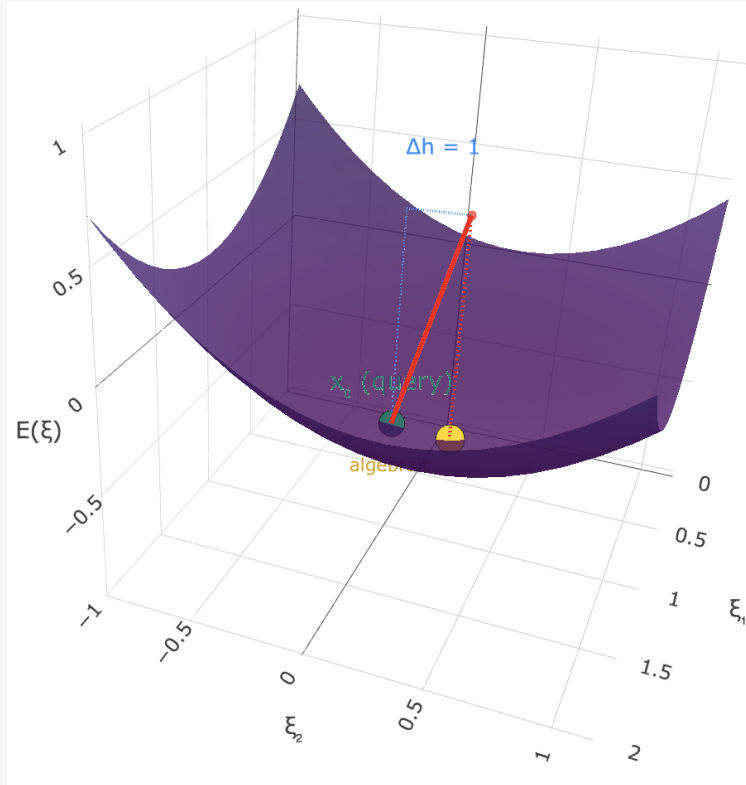
Evolving Energy Landscape:



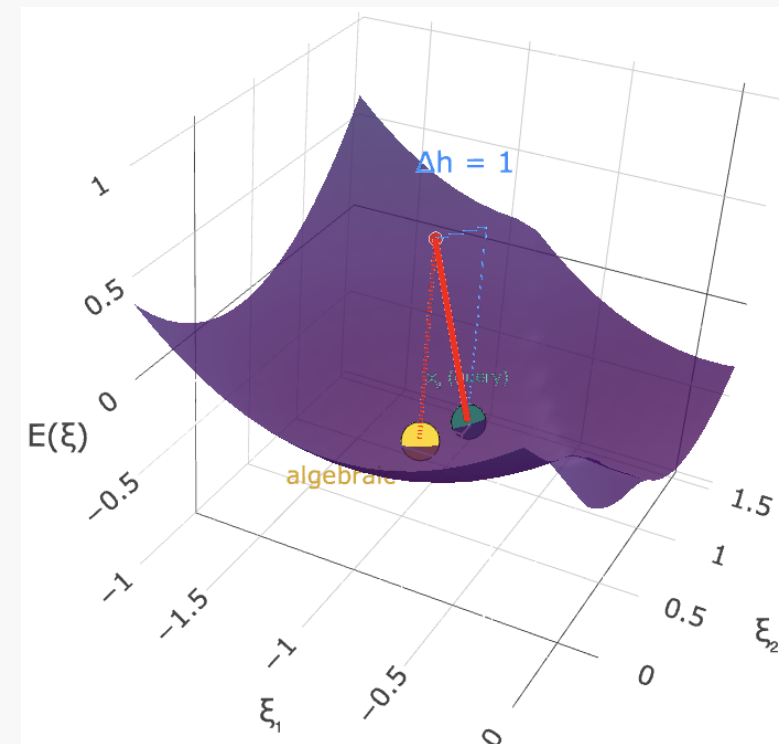
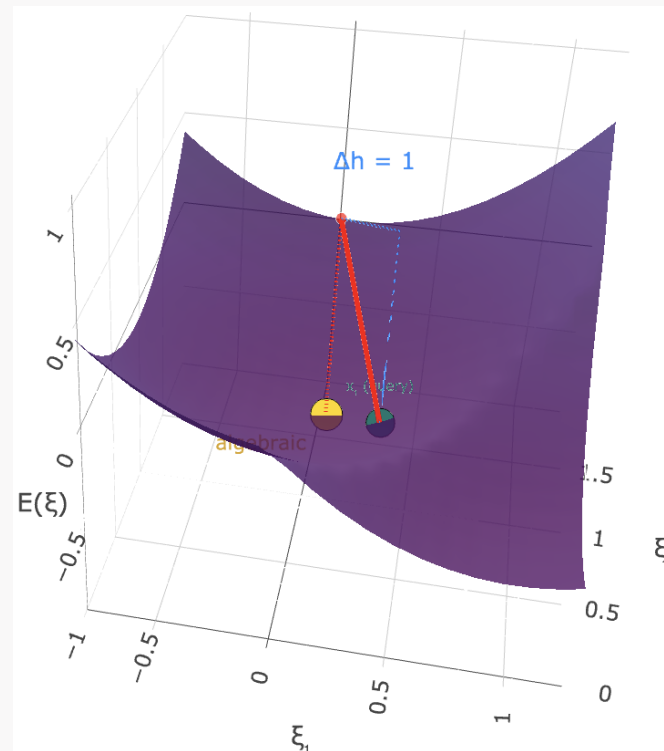
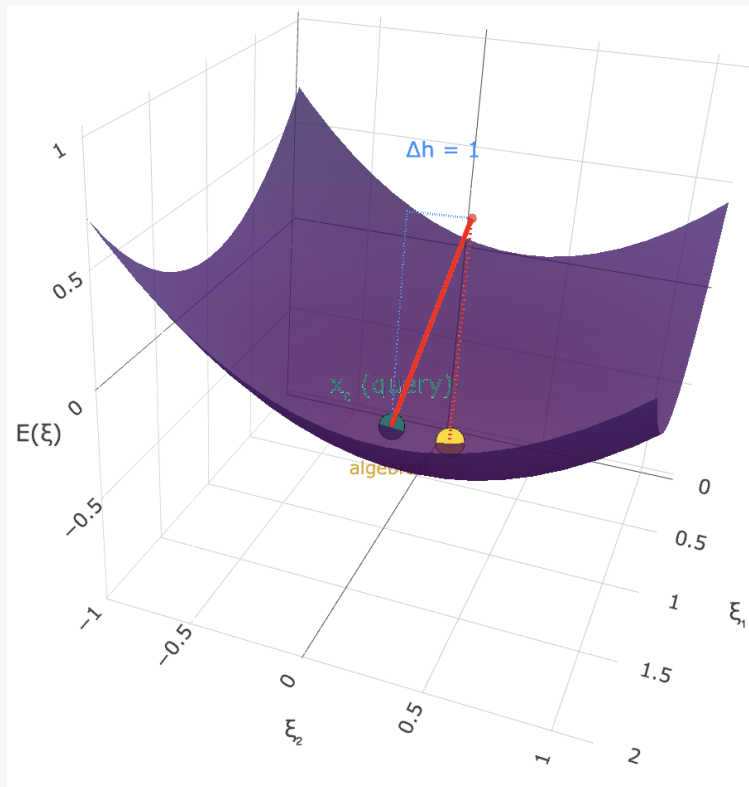
Hopping On the Energy Landscape



Hopping On the Energy Landscape



Hopping On the Energy Landscape



Going beyond the SoftMax

Consider replacing the SoftMax function with other functions— what would change?
For instance, using tanh introduces 2^n modes into the GMM energy landscape across the preceding n tokens.
See the interactive webpage for further details and a live demonstration.

What does these tell us?

Two distinctive inductive biases from the Generalized synchronization mechanism and the Landscape Hopping mechanism.

How Both Mechanisms Shape Inductive Bias

Generalized synchronization: Perturbations in distant history have no effect — the mechanism remains robust and indifferent to them.

Hopping Attention: Noise on tokens is tolerated as long as it leaves the local energy function intact, whether the tokens are from the distant past or the immediate context.

In language tasks, what matters is robustness to meaning — not how far back the perturbation occurred

Example:

Cinderella — “glass” slipper

Change one early word from:

“Cinderella leaves behind a **glass silver** slipper.”

Almost nothing about the story dynamics changes: she still goes to the ball, leaves at midnight, loses a slipper, and the prince searches for her.

But at the end of the story, the identifying object must change:

“the **glass silver** slipper fits. “



In language tasks, what matters is robustness to meaning — not how far back the perturbation occurred

Example:

Cinderella — “glass” slipper

Change one early word from:

“Cinderella leaves behind a **glass silver** slipper.”

Almost nothing about the story dynamics changes: she still goes to the ball, leaves at midnight, loses a slipper, and the prince searches for her.

But at the end of the story, the identifying object must change:

“the **glass silver** slipper fits. “



Though a long-ago word may carry no weight in the immediate story, it can prove critical when referenced later.

With Gratitude:



Stefan Mihalas
@ Allen Institute



Lukasz Kusmierz
@ Allen Institute



Jason Z. Kim
@ Cornell



Danielle Bassett
@ UPenn



Michelle Girvan
@ UMD



Jaideep Pathak
@ NVIDIA



Brian Hunt
@ UMD



Edward Ott
@ UMD